More Parameters Undo What Previous Ones Did: Refining Residual Blocks Indefinitely Without Retraining

Jaume Ivars Grimalt [1]

Abstract

Residual blocks compute $x + f(x)$, the form of one explicit Euler step, yet whether trained residual networks behave as discretised flows remains debated. We refine residual stages at inference—applying each block R times with step $1/R$ so that the integration horizon is conserved—without any retraining. On ConvNeXt, Swin and ResNet the refined stages converge to a continuous limit that costs less than one ImageNet top-1 point, whereas changing the horizon collapses every network. Refinement turns each stage into a high-resolution trajectory on which we measure speed, angular velocity, acceleration and curvature. Comparing a standard ConvNeXt-T with a variant whose nine stage-3 blocks share a single set of weights reveals two regimes behind the same algebra. The shared block is an autonomous flow that searches for a heading, expands along it and refines. The independent blocks form a switched flow: shallow arcs joined by turns of about $80°$ at every block boundary, where almost half of the motion is undone by later blocks. This redundancy explains why the shared stage loses only 0.6 top-1 points with 33% fewer parameters, and suggests that residual depth can do more with fewer, better-used parameters. Code is available at `https://github.com/jaume2000/dynamics_skip_nn`.

1. Introduction

Plain deep stacks such as VGG (Simonyan & Zisserman, 2015) are hard to train as depth grows, partly because gradients vanish or explode through many layers (Bengio et al., 1994; Glorot & Bengio, 2010). Residual connections (He et al., 2016) made very deep networks trainable by letting each block add a correction to its input, $x \mapsto x + f(x)$, which also yields markedly smoother loss landscapes (Li et al., 2018). Because every block writes into the same representation, the stack can be read as a single state that moves through feature space, and the update has the algebraic form of one explicit Euler step of an ODE (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018). That reading inspired Neural ODEs (Chen et al., 2018), but whether a *trained* residual network behaves as an integrator is contested: the analogy can be purely syntactic (Queiruga et al., 2020), and a stack is guaranteed to approach an ODE only if its blocks vary smoothly in depth (Sander et al., 2022).

Two questions follow, and current tools answer neither well. The first is geometric: a stage of nine blocks yields nine states, too few to tell a smooth flow from a sequence of unrelated jumps. The second is about parameters: if each block were one step of a common flow, independent weights per block would be largely redundant. Indeed, a ConvNeXt-T whose nine stage-3 blocks share a single set of weights reaches 80.36% ImageNet top-1, 0.61 points below the untied model with 33% fewer parameters (Section 3.1).

We address both with one inference-time probe. We read each block as a local field integrated over one unit of depth and refine it: apply it R times with step $\Delta t{=}1/R$, so that the stage horizon T is conserved, without any fine-tuning. On ConvNeXt, Swin and ResNet the refined stages converge to a continuous limit that costs less than one top-1 point, whereas changing T collapses every network (Section 3.2). We do not propose refinement as a way to build deeper models; we use it because it turns each stage into a trajectory with hundreds of states per block, on which velocity, angular velocity, acceleration and curvature can be measured.

These trajectories separate two regimes behind the same algebra (Figure 1). The weight-tied stage is an autonomous flow $\dot{x} = f(x)$: it first searches for a heading, then expands along it and finally refines at low speed (Section 3.3). The untied stage is a switched flow $\dot{x} = f_{\lfloor t \rfloor}(x)$: nine shallow arcs joined by turns of about $80°$ at every block boundary. Measured along the path, almost half of the untied motion is cancelled by later blocks, and nearly half of all sign reversals fall exactly on block boundaries, against almost none for the shared field; a frozen decoder can also read the shared state much earlier in the stage (Section 3.4). Much of the untied capacity in depth is thus spent undoing earlier blocks,

[1]Mycrospace AI Research, Valencia, Spain. Correspondence to: Jaume Ivars Grimalt <jaumeig95@gmail.com>.

arXiv preprint.

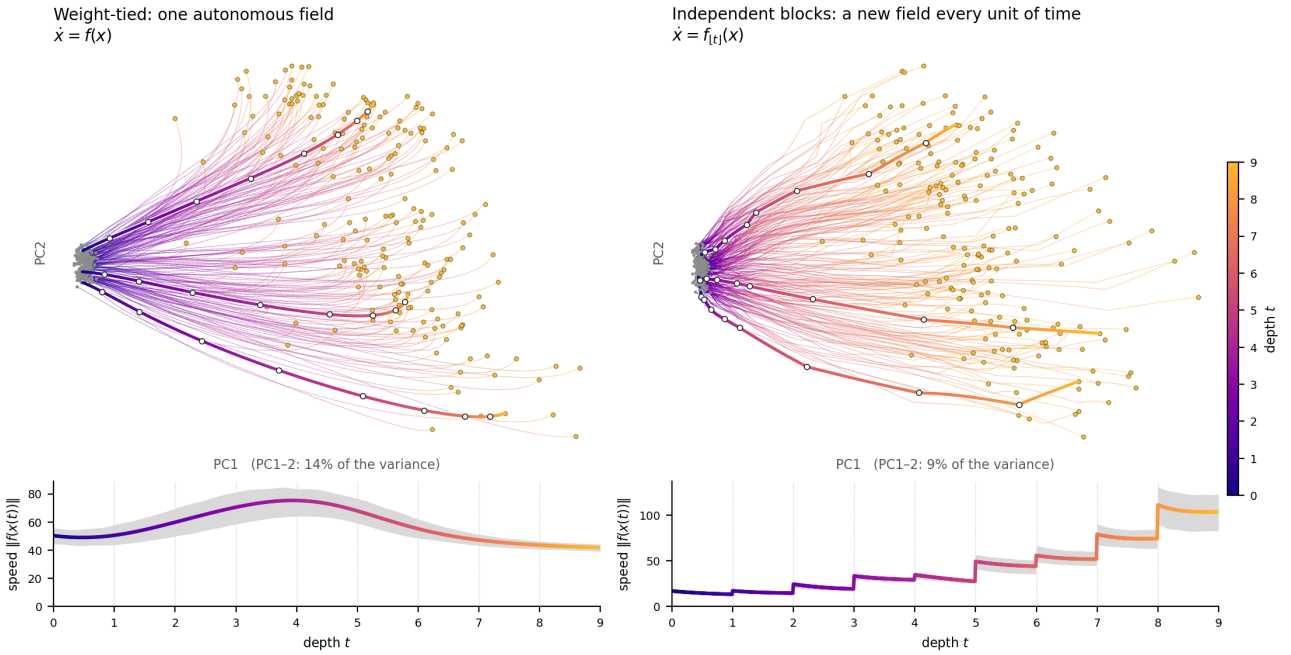


Figure 1. Stage-3 feature trajectories of a weight-tied ConvNeXt (left) and a standard ConvNeXt-T (right), integrated with Euler up to T=9 (Δt=0.09 and Δt=0.01; massive channel removed; 200 ImageNet validation images). **Top:** states are flattened and mapped to 2048 dimensions with a fixed Gaussian random projection (Johnson & Lindenstrauss, 1984; Dasgupta & Gupta, 2003), and PCA is fitted on all projected states of each model (Halko et al., 2011); paths coloured by depth t, white dots mark integer t. **Bottom:** residual speed $\|f(x(t))\|$ (mean over images; shade: P10–P90). One shared block is a single autonomous field $\dot{x} = f(x)$; nine independent blocks switch the field at every integer, $\dot{x} = f_{\lfloor t \rfloor}(x)$. We analyse this contrast throughout the paper.

which is why a single shared block recovers almost all of its accuracy.

Our contributions are:

- **Refinement at fixed horizon** as a training-free probe of residual stages, with a solver-independent continuous limit that stays usable by the downstream network on ConvNeXt, Swin and ResNet (Section 3.2).
- **Autonomous versus switched transport**: a weight-tied stage as a controlled autonomous counterpart of the standard stack, close in accuracy and clearly different in dynamics (Sections 3.1 and 3.3).
- **A kinematic analysis** of feature trajectories (speed, angular velocity, tangential and normal acceleration, curvature), after excluding massive activations that otherwise dominate the statistics (Section 3.3).
- **A measurement of depth redundancy**: cancelled motion, sign reversals at block boundaries, and prefix decoding with the frozen downstream network (Section 3.4).

2. Related Work

Residual networks as discretised ODEs. A residual update $x + f(x)$ (He et al., 2016) is one explicit Euler step of an ODE (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018), a reading that led to Neural ODEs (Chen et al., 2018), multi-level training schemes (Chang et al., 2018) and continuous-in-depth architectures (Queiruga et al., 2020). Whether a trained ResNet actually behaves as such a discretisation is debated. Queiruga et al. (2020) find that standard ResNets fail to act as meaningful integrators beyond the syntactic analogy; Sander et al. (2022) show that the gap between a ResNet and its Neural ODE need not vanish with depth unless the residual maps vary smoothly in depth; Cohen et al. (2021) find that, depending on the architecture and on how trained weights scale with depth, the deep limit can be an ODE, a stochastic differential equation, or neither; and Marion et al. (2024) prove that, under gradient flow and in the large-depth limit, a ResNet initialised as a Neural ODE discretisation remains one through training. These works either train continuous models or study the limit of growing depth. We instead refine *already trained*, standard networks at inference, keeping the integration horizon fixed, and use the refined path as a probe rather than as a model.

Weight sharing and looping in depth. Sharing one residual block across depth links ResNets to recurrent networks (Liao & Poggio, 2016); sharing convolutions across the blocks of a stage yields compact CNNs (Boulch, 2017), as does soft parameter sharing across layers (Savarese & Maire, 2019). In transformers, universal and looped transformers reuse a single layer or a fixed block of layers in a loop (Dehghani et al., 2019; Giannou et al., 2023), ALBERT shares all layers (Lan et al., 2020), and partial sharing schemes trade parameters against accuracy (Takase & Kiyono, 2023; Bae et al., 2025). Iterating a shared block beyond its training depth is fragile: recurrent networks "overthink" and degrade unless they recall the input and are trained with a randomly sampled number of iterations (Bansal et al., 2022); recurrent-depth language models that inject the input at every step and sample the iteration count during training remain stable far beyond their typical depth (Geiping et al., 2025); and deep equilibrium models replace the iteration by a fixed point (Bai et al., 2019). For us weight tying is not a compression method but a control: it turns a switched stack of fields into a single autonomous one, and its trajectory explains both why a shared stage is nearly as accurate and why integrating past the trained horizon fails.

Redundancy and iterative refinement across depth. Residual networks are robust to dropping or reordering blocks: they behave like ensembles of shallow paths (Veit et al., 2016), can be trained with stochastic depth (Huang et al., 2016), and have been read as iterative estimators that refine one representation (Greff et al., 2017; Jastrzębski et al., 2018). Language models show a similar tolerance: middle layers can be skipped, swapped or run in parallel with graceful degradation, although repeating a single middle layer is harmful (Sun et al., 2025; Lad et al., 2025), and many deep layers can be pruned with a small amount of healing fine-tuning (Gromov et al., 2025). Intermediate states are commonly decoded with linear probes (Alain & Bengio, 2017) or lenses (Belrose et al., 2023). These studies establish that depth is redundant; we measure *where* the redundancy sits, as motion that consecutive blocks undo, concentrated at block boundaries, and decode prefixes with the frozen downstream network rather than a trained probe.

Geometry of representations along depth. Under the manifold hypothesis, deep networks gradually untangle and flatten the data manifold (Bengio et al., 2013; DiCarlo & Cox, 2007; Brahma et al., 2016; Hauser & Ray, 2017; Naitzat et al., 2020; Cohen et al., 2020), and its intrinsic dimension changes systematically with depth (Ansuini et al., 2019; Valeriani et al., 2023). Closer to our measurements, the curvature of representation trajectories decreases in human perception of natural videos (Hénaff et al., 2019) and from early to middle layers of language models (Hosseini & Fedorenko, 2023). Recent work treats the residual stream of transformers as a dynamical system whose velocity and acceleration change across layers and whose trajectories are curved (Fernando & Guitchounts, 2025), and measures trajectory length and turning-angle curvature across layers (Pandey et al., 2026). Those analyses sample one point per layer. Refinement provides hundreds of points per block, so velocity, angular velocity, the tangential/normal split of acceleration and curvature can be estimated within blocks and separated from the jumps between them. Finally, outlier features—massive activations in language models (Sun et al., 2024; 2026) and high-norm tokens in vision transformers (Darcet et al., 2024)—dominate such statistics; we find their analogues in ConvNeXt and Swin and exclude them from the kinematics.

3. Refining Residual Stages

We study residual vision networks as a three-part map. The first part—stem, stage 1 and stage 2—encodes the image into a latent state x_0; the second part—the deepest residual stage—transports x_0 to a state x_T ready for decoding; the third part—stage 4 and the prediction head—maps x_T to class logits. We focus on that middle transport (stage 3 in ConvNeXt (Liu et al., 2022), Swin (Liu et al., 2021) and ResNet (He et al., 2016)). It holds most of the residual depth, keeps a fixed representation and spatial resolution, and is the stage with enough blocks to study dynamics: in ConvNeXt-T, for example, stages 1, 2 and 4 have three blocks each, while stage 3 has nine. ConvNeXt inherits this depth layout from Swin, itself patterned on ResNet, so the same middle stage is the natural object of study in all three.

In ConvNeXt-T we take that stage's $n{=}9$ blocks as the trained horizon $T{=}n{=}9$, and refine the discretisation with repetitions R and step $\Delta t{=}1/R$ (Section 3.2). Effective per-block step sizes may differ, but they are absorbed into the learned weights, so integrating each block over one unit of time is equivalent to the steps the model has already internalised.

The central question is functional: can a weight-tied stage 3—one shared residual field reused over the horizon $T{=}n$—match the representational power of the non-tied stage, which uses n independent blocks and many more parameters? Simply looping the blocks at finer step size does not answer this. Residual maps are Lipschitz, so refining the discretisation at fixed T always converges to a continuous limit, trained or randomly initialised (Appendix D); convergence alone is therefore a weak claim. What the shared construction does give us is a clean autonomous ODE for stage 3. We then read the standard network the same way, block by block: each residual update is a one-step Euler step of a local field, so the untied stage is a switched composition of smooth ODE pieces rather than a single autonomous flow—hence the jumps in $\|f\|$ at block boundaries in Fig-

ure 1. Continuising both—tied and untied—at inference yields high-resolution trajectories from x_0 to x_T, and those trajectories are where the two models diverge: the standard path repeatedly redirects and partially undoes itself, while the tied path stays closer to a monotone transport. We organise the section accordingly:

1. Section 3.1 shows that a weight-tied stage 3 nearly matches an untied ConvNeXt-T (80.36% vs. 80.97% top-1), so one shared field already carries most of the function;
2. Section 3.2 continuises residual stages at inference without fine-tuning—ConvNeXt-T and its weight-tied variant, and also Swin-T and ResNet-50/101—showing the same pattern across architectures: at fixed T every solver reaches one limit and retains almost all accuracy, whereas breaking T collapses the networks;
3. Section 3.3 compares the refined trajectories, autonomous (tied) versus switched (untied);
4. Section 3.4 measures that pull-and-push along depth, and probes when stage 3 prefixes become usable by a frozen stage 4+head.

3.1. Do weight-tied stage-3 networks achieve similar results?

We first ask whether the stage-3 transport needs nine independent blocks, or whether one shared block applied nine times is enough. Sharing weights across depth is not new: residual networks with shared blocks have been linked to recurrent networks (Liao & Poggio, 2016), residual networks with partially shared blocks have been trained as compact CNNs (Boulch, 2017; Savarese & Maire, 2019), and looped or universal transformers reuse a single layer or a fixed block of layers many times (Dehghani et al., 2019; Giannou et al., 2023). Sun et al. (2025) study a related question in pretrained transformers: they skip, reorder, parallelise and repeat middle layers, and find that the middle layers share a representation space but do not implement the same operation—replacing them by repeated copies of the central layer is the most damaging intervention they test. Large language models likewise tolerate deleting or swapping middle layers (Lad et al., 2025) and pruning a large fraction of the deepest ones after light fine-tuning (Gromov et al., 2025). Analogous ablations on ConvNeXt stage 3 (permuting, duplicating or substituting blocks) are reported in Appendix E. Reusing blocks that were trained to be different tests the wrong object, however; we instead ask whether a stage *trained* to use a single shared function reaches a similar accuracy level.

We train a vanilla ConvNeXt-T and a weight-tied stage-3 variant under the same recipe (ImageNet-1k, batch size 1024; configuration in Appendix A). Our baseline reaches 80.97% top-1 (28.59M params), slightly below the original ConvNeXt-T report of 82.1% (Liu et al., 2022)—expected under our hardware constraints—but all comparisons below use models trained by us under matched conditions.

The weight-tied model (18.97M params, 33% fewer) reaches 80.36% top-1 (-0.61 points). Sharing all four stages (8.68M params, 70% fewer) reaches 79.01% (-1.96 points).

A network that implements (approximately) the same residual function at every depth can therefore sit close to the untied baseline. That motivates the next subsection—not as a recipe for a deeper or larger network, but as an inference-time probe: if stage 3 is (approximately) an Euler discretisation, refining it with step $\Delta t{=}1/R$ at fixed $T{=}9$ should keep a decoder-usable state and, equally important, yield high-resolution trajectories for analysis. We apply that probe to the weight-tied model and to standard residual architectures.

3.2. Indefinite looping without fine-tune

Section 3.1 showed that one shared block nearly matches a standard ConvNeXt stage 3 (independent per-block weights). We now refine residual blocks at inference as one-step Euler updates of a vector field: repeat each update R times with step $\Delta t{=}1/R$, keep the trained horizon $T{=}n$, and do not fine-tune. The goal is not to replace a small network by a R-times deeper one that trades a fraction of a point of accuracy for width in depth—that would be the wrong reading. Refinement is a probe with two uses. First, it tests whether the continuous limit stays functionally close to the native model for stage 4 and the head: on ConvNeXt-T the refined endpoint sits $\sim 21^\circ$ from the $R{=}1$ state (Figure 5), yet top-1 barely moves, so the decoder still sees a usable representation. Second, it raises the temporal resolution of the path from x_0 to x_T, which is what later geometry and undoing analyses need. Convergence of the refinement by itself is not diagnostic—any Lipschitz residual has a continuous limit (Appendix D)—so what matters is what the limit retains and how the trajectory is organised. We apply the probe to weight-tied and standard networks (ConvNeXt, Swin, ResNet); trajectory geometry is deferred to Section 3.3.

A residual block computes $x + f(x)$ (He et al., 2016). With step size 1, that is exactly one step of the explicit Euler method for the ODE $\dot{x} = f(x, t)$ (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018), whose continuous-depth limit is a Neural ODE (Chen et al., 2018). Write the residual of block i as

$$f_i(x) \;=\; \mathrm{block}_i(x) - x. \tag{1}$$

Then the native network is one Euler step of size 1 per block.

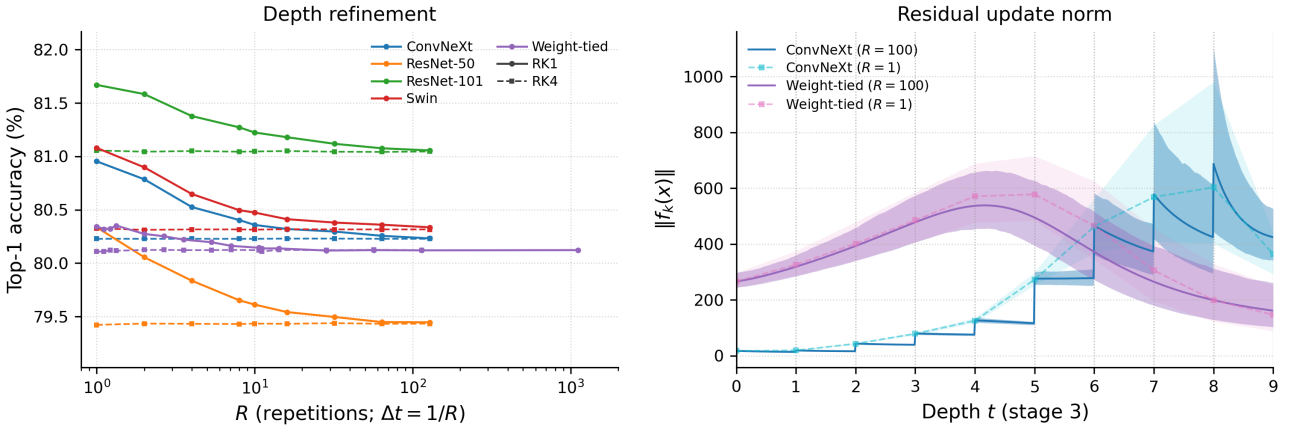


Figure 2. **Left:** top-1 vs. repetitions R (solid: Euler / RK1; dashed: RK4), with $\Delta t{=}1/R$ so $T{=}n$ is conserved (weight-tied: nR applications of the shared block). **Right:** residual-field norm $\|f(x)\|$ along stage 3 (mean and P5–P95 over 500 images). ConvNeXt is piecewise constant in depth; the weight-tied model is a single smooth autonomous field. See Section 3.2.2.

Over a stage of n blocks (e.g. $n{=}9$ in ConvNeXt-T stage 3), starting from the output x_0 of the previous stage,

$$x_{i+1} \;=\; x_i + f_i(x_i), \qquad x_n \;=\; x_0 + \sum_{i=0}^{n-1} f_i(x_i). \quad (2)$$

These works interpret the residual update as a forward-Euler step (E, 2017; Haber & Ruthotto, 2017; Lu et al., 2018), but algebraic form alone is not sufficient evidence that a trained network behaves as an Euler solver: Queiruga et al. (2020) argue that ResNets fail to be meaningful dynamical integrators beyond that syntactic similarity, and Sander et al. (2022) show that the gap between a ResNet and its associated Neural ODE need not vanish with depth unless the residual maps vary smoothly in depth (otherwise the discrete stack need not be a consistent discretization of a single continuous flow); Marion et al. (2024) prove that, under gradient flow and in the large-depth limit, a ResNet initialised as such a discretization keeps that property through training.

We suggest reading the stage as a *sum of several ODEs*, one per block, where the network is trained to solve each unit interval with a single Euler step. The continuous object behind (2) is therefore

$$x_n \;=\; x_0 + \sum_{i=0}^{n-1} \int_i^{i+1} f_i\big(x(t)\big)\, dt, \quad (3)$$

or, block by block,

$$x_{i+1} \;=\; x_i + \int_i^{i+1} f_i\big(x(t)\big)\, dt. \quad (4)$$

Here the sum is the stack of residual blocks, and each integral is the continuous counterpart of the one-step update in (2): instead of taking one Euler step of size 1, we integrate f_i over the unit interval $[i, i+1]$, keeping the horizon of each block equal to 1. For a stage with n blocks trained at unit step, the stage horizon is therefore

$$T \;=\; n \qquad \text{(ConvNeXt-T stage 3: } n{=}9\text{, so } T{=}9\text{)}. \quad (5)$$

Repetitions R. Throughout the paper we use a single refinement parameter R (*repetitions*). For a non-tied stage, $R{=}2$ means each of the n blocks is applied twice, for nR residual updates in total. For a weight-tied stage, $R{=}2$ means the *same* shared block is applied $nR{=}18$ times. In both cases we set the Euler step from the trained horizon,

$$\Delta t \;=\; \frac{1}{R} \qquad \Rightarrow \qquad T \;=\; nR \cdot \Delta t \;=\; n, \quad (6)$$

so refining never changes the integration time—only the resolution.

Plain interpolation keeps the field piecewise constant on each block and discretizes the integral with R Euler substeps of size $\Delta t = 1/R$:

$$\int_i^{i+1} f_i\big(x(t)\big)\, dt \;\approx\; \sum_{r=0}^{R-1} \Delta t\, f_i\big(x_{i+r\Delta t}\big). \quad (7)$$

Equivalently, over the whole stage the refined trajectory is

$$x_T \;=\; x_0 + \sum_{i=0}^{T-1} \sum_{r=0}^{R-1} \Delta t\, f_i\big(x_{i+r\Delta t}\big), \qquad \Delta t = \tfrac{1}{R}. \quad (8)$$

When $R{=}1$, (8) recovers the original network (2). As R grows with $T{=}n$ fixed, the scheme approaches the continuous limit (3); we also compare against Midpoint (RK2) and RK4 on the same horizon.

In a weight-tied stage the residual field is the same at every depth, $f_i \equiv f$ for all i. Then (3) collapses to a single autonomous integral over $[0, T]$ with $T{=}n$:

$$x_T \;=\; x_0 + \int_0^T f\big(x(t)\big)\, dt. \tag{9}$$

Inference uses nR applications of f with $\Delta t{=}1/R$; training uses $R{=}1$ (n steps of size 1), hence $T{=}9$ for our shared ConvNeXt-T.

3.2.1. Ablations

We apply the refinement (8) (plain interpolation, T conserved via $\Delta t{=}1/R$) to our ConvNeXt-T, our weight-tied ConvNeXt-T, Swin-T, ResNet-50 and ResNet-101 (ImageNet-1k validation, top-1). Figure 2 (left) summarises the sweep over R; numbers are in Table 1.

Accuracy decreases monotonically as R grows and plateaus at a continuous limit that matches RK4. Euler (RK1), Midpoint (RK2) and RK4 all converge to the *same* accuracy once the step is fine enough; full Mid/RK4 tables are in Appendix B. (Matched top-1 alone does not prove a shared state x_T; we check endpoint agreement of the solvers later in Appendix C.) The trained network itself (Euler, $R{=}1$) is *not* at that limit; it is the best point of every sweep. Keeping the nine steps of size $\Delta t{=}1$ but swapping Euler for RK4 already lands on the limit, both in accuracy and in the stage-3 state (Appendix C), and costs 0.6–0.9 points on the untied models (0.2 on the weight-tied one): the decoder is tuned to the state produced by a single Euler step of size 1, not to the flow it approximates.

Wrong step size. Repeating blocks without shrinking the step (e.g. $R{=}10$ with $\Delta t{=}1$, so T grows tenfold) collapses every pretrained model to chance ($\approx 0.1\%$–1% top-1). The same holds for the weight-tied network when T leaves the trained horizon (Figures 3 and 4). The Euler step must be $\Delta t{=}1/R$, equivalently $T{=}n$ for an n-block stage ($T{=}9$ when $n{=}9$). Recurrent networks iterated past their training depth show the same degradation, termed overthinking (Bansal et al., 2022); recurrent models that unroll to arbitrary depth at test time are trained with a randomly sampled number of iterations (Bansal et al., 2022; Geiping et al., 2025), whereas our blocks were trained for one horizon. The trajectory dynamics behind this collapse are analysed in Section 3.3.5.

Weight-tied fractional $R < 1$. Figure 3 reports the shared network at fixed $T{=}9$ with fewer than nine steps ($R < 1$, i.e. $9R$ applications with $\Delta t{=}1/R > 1$); the numbers, including Midpoint, are in Table 9. Large steps need a higher-order solver: at $R{=}4/9$ (4 steps), RK4 reaches 78.9% while Euler falls to 67.8%; Euler with $R{=}2/9$ ($\Delta t{=}4.5$) is unstable (5.9%). For $R \geq 7/9$, all three solvers sit within 0.3 points of the trained baseline.

Table 1. Plain Euler refinement with $\Delta t{=}1/R$ (T conserved). Top-1 (%). For weight-tied, R means $9R$ applications of the shared block (exact $9R$ when available; else nearest measured depth*). ODE limit: RK4 at the finest step in each sweep.

R (Δt)	CNX-T	Swin-T	R-50	R-101	Tied
1 (1)	80.95	81.08	80.34	81.67	80.34
2 (0.5)	80.79	80.90	80.06	81.58	80.27
4 (0.25)	80.53	80.65	79.84	81.38	80.22*
10 (0.1)	80.36	80.47	79.61	81.22	80.14*
32 (0.03)	80.30	80.38	79.50	81.12	80.14*
128 (0.008)	80.23	80.34	79.45	81.06	80.12*
ODE limit	80.23	80.32	79.43	81.05	80.12
Gap ($R{=}1 \to$ODE)	-0.72	-0.76	-0.91	-0.62	-0.22

*Nearest measured shared depth at $T{=}9$: 32, 96, 128, 1024 steps respectively (exact $9R$ would be 36, 90, 288, 1152).

Horizon, not resolution. Varying the number of unit steps at $\Delta t{=}1$ (so T grows) or varying Δt at fixed $R{=}1$ (9 steps; $T{=}9 \cdot \Delta t$) both peak at $T{=}9$ (Figure 4): the classification head expects the state reached after that integration time. Integrating too short or too long degrades smoothly near $T{=}9$ and collapses far from it. We conclude that the best Euler step is the one that preserves the trained horizon: $\Delta t{=}1/R$ for every model (non-tied and weight-tied).

Do refined blocks land on the same x? Near-baseline accuracy does *not* mean that R Euler steps of size $1/R$ reproduce the unit-step state. Figure 5 compares the refined ConvNeXt-T trajectory ($R{=}100$, $\Delta t{=}0.01$) with the native $R{=}1$ trajectory on the same images, in ℓ_2 distance (top, next to both state norms) and cosine similarity (bottom) (massive channel removed from the metrics, see Section 3.3.1; $N{=}500$ images). Three facts stand out.

- **Within each block** the refined trajectory moves *toward* the parallel $R{=}1$ checkpoint: the distance decays and the cosine rises until the block ends, so the refinement is not drifting away from the one-step endpoint of that block.
- **Across blocks** the gap grows: the block-end distance rises from ≈ 5.6 to ≈ 76 at $t{=}9$, against $\|x\| \approx 203$ at the stage exit, i.e. a relative gap $\|x^{(R)} - x_{R=1}\|/\|x\| \approx 0.36$.
- **The gap is a rotation.** The block-end cosine drops from 0.99 to 0.93 ($\sim 21°$), while $\|x\|$ differs by only $\approx 3\%$ (203 vs. 210); a pure rescaling of that size would account for a distance of ≈ 7, not 76.

Each block therefore implements a *different* map than its one-step counterpart once the input has already drifted; local

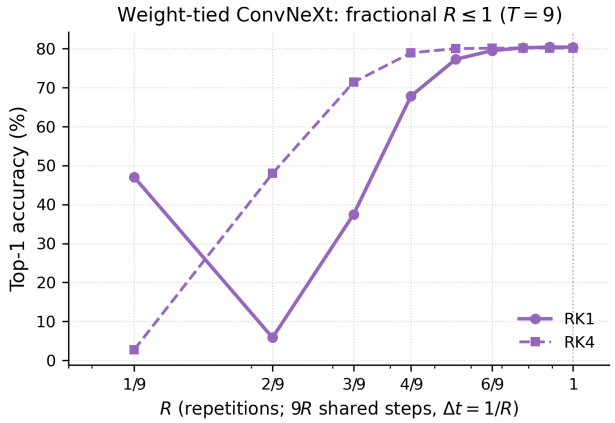


Figure 3. Shared ConvNeXt-T at fixed T=9 for fractional $R \le 1$ (RK1 solid, RK4 dashed).

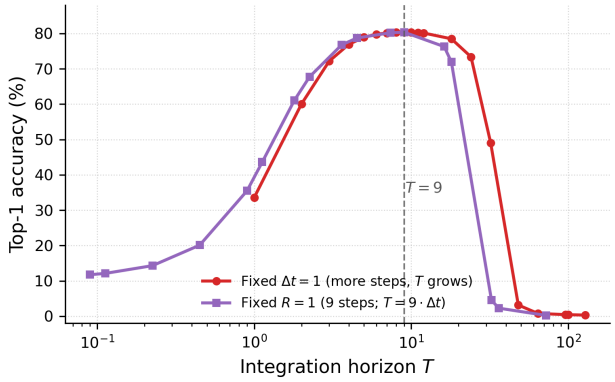


Figure 4. Shared ConvNeXt-T: top-1 vs. horizon T. Red: fixed Δt=1, more steps (T grows). Purple: fixed R=1 (9 steps), varying Δt (T=9 · Δt). Both peak at the trained horizon T=9.

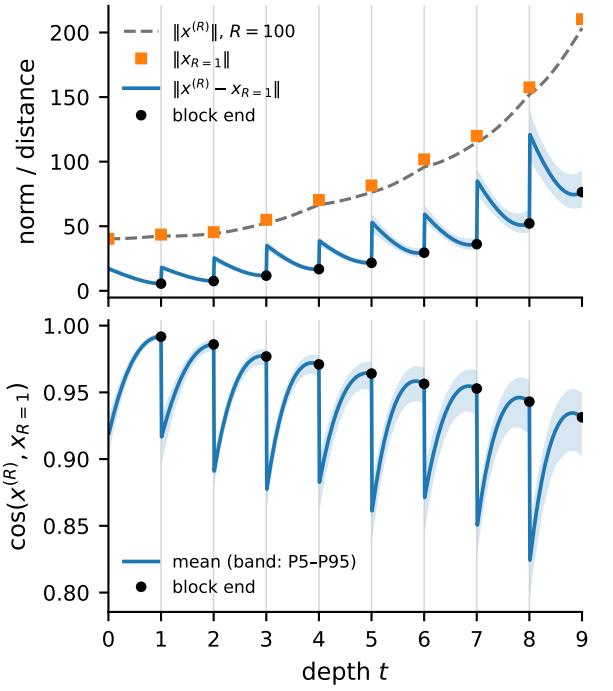


Figure 5. ConvNeXt-T, R=100, Δt=0.01, against the native R=1 path. **Top:** $\|x^{(R)}(t) - x_{R=1}\|$ (solid), with the state norms of both paths for scale (dashed: R=100; squares: R=1). **Bottom:** $\cos(x^{(R)}, x_{R=1})$. Mean and P5–P95 over 500 images; massive channel ignored; dots mark block ends. The norms nearly coincide while the block-end gap grows with depth: the mismatch is mostly in angle.

refinement stays contractive toward the current block's R=1 target, but errors accumulate through the switched stack and the final x_T points in a different direction from the native one. Nevertheless, that displacement—$\sim 21°$ on ConvNeXt-T, $\sim 11°$ on the weight-tied model (Section 3.3.4)—still lands in a zone that stage 4 and the head can classify: without training, refining at fixed T costs only 0.6–0.9 top-1 points on untied models and 0.2 on the weight-tied one (Table 1). The next paragraph asks whether a light finetune of that decoder closes the rest (already yes at R=3; R=100 pending).

Finetuning stage 4 and the head after unrolling. As a complementary check, we freeze stem and stages 1–3 and finetune only stage 4 and the classification head on the unrolled transport (horizon T=9 conserved). On ConvNeXt-T with R=3 (Δt=1/3) this recovers 80.93% top-1—within 0.04 points of the untied baseline (80.97%). For contrast, the same R=3 with Δt=1 (broken horizon T=27) collapses to 41% *without* finetuning. That recovery indicates that the residual gap is mostly a decoder mismatch—stage 4 and the head were trained on the native R=1 landing zone, not on the refined x_T—rather than a collapse of class information in the transport itself. The same protocol at the fine step of Figure 5 yields XX% / XX% for ConvNeXt-T / weight-tied at R=100 (10 epochs, learning rate 10^{-6}).

Takeaway. Refinement at fixed T yields a real continuous limit—solver-independent (Appendix C), present even for random weights (Appendix D)—that is *not* the native R=1 state, yet remains decoder-usable and nearly recoverable by adapting stage 4 and the head. What training selects is therefore not convergence itself, but *how* the path is built: one shared block throughout the stage versus a switched stack of one-shot blocks. Section 3.3 contrasts those two regimes.

Continuous-in-depth models express the weights as a function of depth in a basis (Queiruga et al., 2020); the piecewise-

constant basis recovers the ResNet, and smoother bases are a natural alternative. We tried the simplest one, replacing the piecewise-constant schedule $\theta_t = \theta_{\lfloor t \rfloor}$ by a bilinear blend of consecutive block weights. It performs relatively badly on every architecture (e.g. -6.6 points on ConvNeXt-T at R=128 vs. plain); full numbers and the residual-norm trajectories are in Appendix F.

3.2.2. Piecewise vs. smooth residual fields

Figure 1 showed the two regimes on individual trajectories; Figure 2 (right) makes the same contrast on the residual-field norm over 500 images. In the non-tied ConvNeXt, $\|f_k(x)\|$ is approximately constant inside each block and jumps at integer depths: a *smooth function by parts*, i.e. an autonomous field on each unit interval $[i, i+1]$ with discontinuous switches of θ between blocks (Equation (3)), a switched system in the control-theory sense (Liberzon, 2003). In the weight-tied model the same shared f is applied for the whole stage, so the residual norm evolves as one continuous curve that rises and then decays (Equation (9)). Markers at R=1 sit on those curves; refining to R=100 fills in the intra-block path without changing its global shape. This is the geometric counterpart of the accuracy tables: refining a switched stack converges to a slightly worse continuous limit, while refining a shared field stays near the trained accuracy as long as T is held fixed. Sections 3.3 and 3.4 analyse these trajectories in more detail—in particular, how untied blocks redirect the state between integer depths even though each intra-block piece remains integrable.

3.3. Geometrical analysis of feature trajectories

We now compare the geometry of stage-3 trajectories under a single shared field versus nine independent blocks. The weight-tied ConvNeXt traces a smooth three-phase arc over T=9. The untied ConvNeXt-T—and likewise ResNet and Swin—does not: each block learns a one-step map, so the refined path is a polyline with a new heading at every integer.

3.3.1. Massive activations excluded from analysis

While inspecting the trajectories obtained with our first approach, we noticed that those of ConvNeXt and Swin were unusually homogeneous. Analysing the features revealed massive activations (Sun et al., 2024). In both the shared and the untied ConvNeXt a single stage-3 channel exceeds all others by more than an order of magnitude and keeps growing in magnitude along the stage, a channel-concentrated pattern also reported for diffusion transformers (Gan et al., 2025). In Swin the outliers are instead the four corner tokens, whose norms end the stage more than an order of magnitude above those of typical positions, reminiscent of the high-norm tokens of vision transformers (Darcet et al., 2024). ResNet stage 3 shows no comparable outlier. This is consistent with Sun et al. (2026), who find that massive activations arise in pre-norm residual blocks and are suppressed when the vector-wise normalisation is replaced by an element-wise one: ConvNeXt and Swin normalise each position across channels with LayerNorm, whereas BatchNorm (Ioffe & Szegedy, 2015) in ResNet normalises each channel independently. Details and figures for all models are given in Appendix G.

The trained ConvNeXt relies on this channel: zeroing it after every residual step of the shared model reduces ImageNet top-1 from 80.3% to 67.8%, whereas zeroing a random stage-3 channel costs at most 0.3 pp. This does not make the channel necessary, since zeroing it also changes the LayerNorm statistics of every position, and in LLMs massive activations can be removed through architectural changes without loss of performance (Sun et al., 2026).

Unlike in LLMs, where massive activations are injected by a few early blocks and then stay nearly constant until late blocks cancel them (Sun et al., 2026), the ConvNeXt channel evolves throughout the stage with the same kinematic profile as the rest of the state (Figure 13). In LLMs these activations act as fixed, input-independent bias terms, i.e. implicit parameters of the network (Sun et al., 2024; 2026), but in the shared model the explicit parameters are identical at every step, so the blocks can only access depth through the state itself. We conjecture that this channel provides that signal, in the same way that an autonomous system can emulate a non-autonomous one by carrying time as an extra state variable (Strogatz, 2015), which time-dependent Neural ODEs implement explicitly as an extra input channel (Dupont et al., 2019). Our observations are consistent with this reading but do not establish it; a dedicated study comparing the dynamics with and without the channel is left for future work.

3.3.2. Trajectories of weight-tied ConvNeXt

We report the same four kinematics on every model, ignoring the massive ConvNeXt channel and the massive Swin tokens (ResNet has neither). We measure velocity $\|v\|$, angular velocity ω, the tangential/normal split (a_t, a_n) of acceleration, and curvature $\kappa = a_n/\|v\|^2$ (Figure 6; N=500). A turning-angle curvature between consecutive layers has been used for language models (Pandey et al., 2026), and curvature of representation trajectories has been used to show that human visual perception (Hénaff et al., 2019) and language models (Hosseini & Fedorenko, 2023) straighten their inputs; there it is measured across the input sequence (frames or tokens), here along depth for a fixed input. Speed says how fast the state moves; ω and κ say how much its heading bends. Here $\kappa = \mathrm{d}\theta/\mathrm{d}s$ is Frenet curvature of the heading of v: $\kappa = 0$ is a straight path, $\kappa \to \infty$ a corner. The

number is in $1/\text{length}$, so it is read against the arc travelled per unit depth, $s \approx \|v\|\Delta t$. Then $\Delta\theta \approx \kappa\|v\|\Delta t$ (equivalently $\omega\Delta t$). A value of order 1 with $\|v\|$ of tens would be a hairpin per unit of depth; $\kappa \sim 10^{-2}$ still turns the heading by tens of degrees over a block.

Read together, the shared curves in Figure 6 (top) are a single arc with three phases, not a catalogue of independent plots.

Early on, $\|v\|$ is still moderate while ω and κ peak ($\omega{\approx}0.70\,\text{rad}/t$, $\kappa{\approx}1.4{\times}10^{-2}$). Per unit of depth that is $\Delta\theta \approx \kappa\|v\| \approx 0.7\,\text{rad}$ ($\approx 40°$): small steps that keep changing direction, as if committing to an axis.

Through the first half of the stage that axis is used. The signed tangential acceleration $a_t = \langle\hat{v}, a\rangle$ turns positive, peaks near $t{\approx}2.3$, and $\|v\|$ follows a bell, as in Figure 2 (right) and Figure 1 (bottom left). Meanwhile ω and κ fall, and κ reaches its minimum ($\approx 4{\times}10^{-3}$, $\Delta\theta \approx 19°$ per unit of depth) exactly when a_t crosses zero at mid-horizon $t{=}4$: the path is at its straightest while features of different classes are pushed apart. That single tangent direction still carries on average $\approx 27\%$ of $\|a\|$, a large share in a $\sim 70{,}000$-dimensional state ($384{\times}14{\times}14$); the rest is $a_n > 0$, so the trajectory never becomes purely one-dimensional.

After $t{=}4$ the same geometry returns as braking rather than search: a_t is negative (most so near $t{\approx}5.3$), $\|v\|$ decreases, and ω and κ rebound (secondary peaks near $t{=}6$–6.5). Total $\|a\|$ declines overall ($\approx 34 \to 8$) with a rebound while the field brakes.

At $t{=}9$, $\|a\|$ is small, $\|v\|$ remains finite, and both ω and κ are low but non-zero ($\approx 0.20\,\text{rad}/t$, $\approx 4.8{\times}10^{-3}$). This is a coasting state, not an identity map: features keep drifting, the regime of Section 3.3.5.

Therefore the shared stage can be read as three jobs in one field. At the start the model searches for a heading: $\|v\|$ is still moderate, so the steps are small, while ω and κ are high and the tangent turns by tens of degrees per unit of depth. It then accelerates along that heading more nearly in a straight line ($a_t > 0$, ω and κ fall, $\|v\|$ peaks): this is the expansion of class features. Finally it brakes and turns again with small steps ($a_t < 0$, $\|v\|$ drops, ω and κ rebound)—a refinement of the same geometry as the search, not a new block. Over the 500 images this profile is typical; $\|v\|$ varies most in the expansion phase, where classes separate along different directions.

3.3.3. Trajectories of untied ConvNeXt

The untied ConvNeXt-T has no such arc (Figure 6, bottom). Each block is a similar one-step map; the next block replaces the vector field and sends the state into a different direction. ResNet and Swin look the same. At every integer, v jumps, so finite-difference ω, a and κ diverge as $\Delta t \to 0$. Those inter-block pairs are omitted from the figure. The reported $\omega{\sim}130$–$150\,\text{rad}/t$ is not a turn of a hundred radians: it is $\arccos\langle\hat{v}_d, \hat{v}_{d+1}\rangle/\Delta t$ on a *single* micro-step of $\Delta t{=}0.01$, so the heading jump between blocks is $\omega\Delta t \approx 1.3$–$1.5\,\text{rad}$ ($\approx 75°$–$86°$). Dividing a finite angle by a vanishing step is why the spikes explode; they only record that neighbouring blocks do not share a direction. What remains is the within-block motion, which is the fair comparison to the shared flow.

Inside a block the dynamics are short. $\|v\|$ is set by the new field and then decays slightly ($a_t < 0$ on most of the interval; a small positive tail appears only in the last blocks). The envelope of $\|v\|$ grows in steps ($\approx$17 after the first block, $\approx$112 at the start of the last), not as a bell. After each switch, ω starts high (≈ 0.6–$0.9\,\text{rad}/t$) and falls toward ≈ 0.45–0.55: a brief reorientation, then a more stable heading. As in the shared model a_n dominates ($\approx 97\%$ of $\|a\|$), so the segment still bends, but there is no stage-wide accelerate-then-brake cycle.

That bending is not a hairpin and not a straight line. Early intra-block $\kappa{\approx}4{\times}10^{-2}$ with $\|v\|{\approx}17$ is *larger* than the shared peak, again about $40°$ of heading per unit of depth; by the last block κ has fallen to $\approx 5{\times}10^{-3}$ (still $\sim 30°$ per block, because $\|v\|$ is larger), matching the shared *minimum*. Relative to the $80°$ kinks at the integers the segments look nearly linear; relative to a Euclidean straight line they still turn tens of degrees.

Therefore the picture to keep in mind is not a semicircle and not the shared three-phase arc. It is a chain of nine similar strokes, each pointing a different way. Within a stroke the heading changes by a comparable angle ($\sim 30°$–$40°$): most of the turn sits at the start of the block, where ω is high, and then the path flattens—a shallow arc, closer to a stretched parabolic segment than to a circular quarter. Because $\|v\|$ grows with depth, later strokes have the same order of heading change but a longer arc: the pieces get larger, not more tightly wound. At every integer those arcs are joined by an $\approx 80°$ kink, where the next block replaces the field. The refined path is that polyline, not a search–expand–refine flow over $T{=}9$. That geometry is what Section 3.4 measures as consecutive-block cosine 0.17 and undo fraction 0.25. A deeper untied stack need not grow a shared-style flow: independent residual blocks already behave like ensembles of short paths (Veit et al., 2016).

3.3.4. Refinement error: accumulated vs. bounded

Figure 7 repeats the comparison of Figure 5 for the weight-tied network. Both models start the same way: after the first native step the cosine to the $R{=}1$ path is 0.99. They

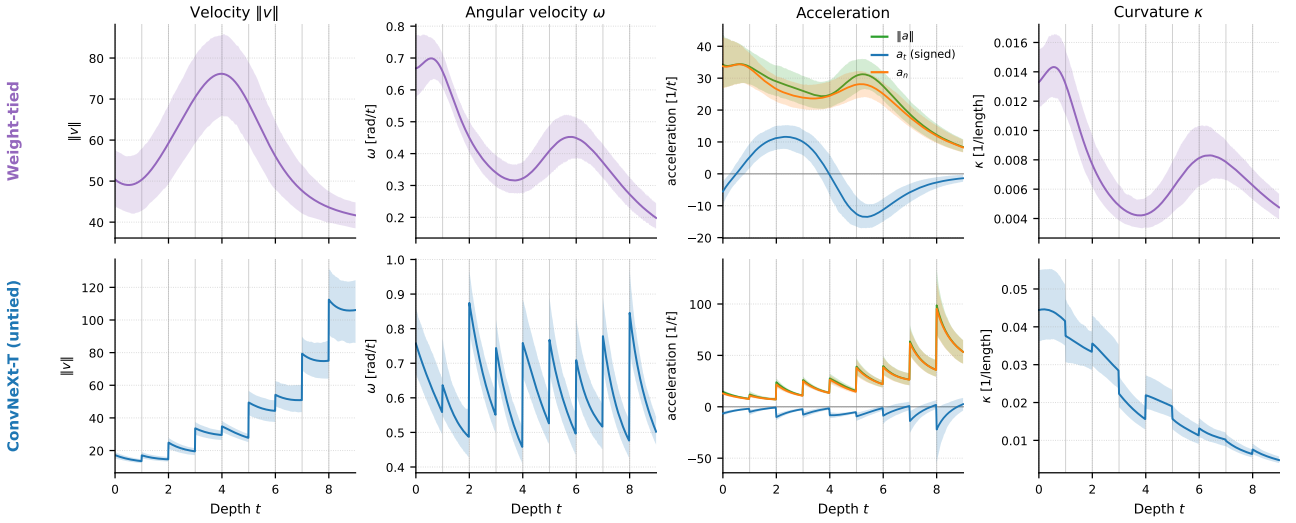


Figure 6. Stage-3 kinematics over $T{=}9$ ($R{=}100$, Euler $\Delta t{=}0.01$; massive channel ignored; mean and P5–P95 over $N{=}500$). **Top:** weight-tied ConvNeXt. **Bottom:** untied ConvNeXt-T. Columns: velocity $\|v\|$, angular velocity ω, acceleration decomposition $(\|a\|, a_t, a_n)$, curvature $\kappa{=}a_n/\|v\|^2$. The shared field traces a smooth three-phase arc; the untied stack is piecewise within each block. On the untied row, pairs that straddle a block boundary are omitted from ω, a and κ: there v is discontinuous, so finite differences diverge as $\Delta t \to 0$ and are not representative of the within-block dynamics.

then diverge. In the standard ConvNeXt-T the block-end cosine keeps falling, to 0.93 at $t{=}9$: each block receives an input that has already drifted and adds its own discretisation error on top. In the weight-tied model it settles at ≈ 0.98 after two or three steps and stays there for the rest of the stage, so a single autonomous field keeps the refinement error bounded instead of accumulating it. The norms also behave differently: in ConvNeXt-T they nearly coincide (203 vs. 210), whereas the weight-tied refined path ends about 7% shorter (427 vs. 458), so part of its smaller gap is radial.

3.3.5. Integrating beyond the trained horizon

At the end of the trained horizon the shared field has small acceleration but non-zero velocity (Section 3.3.2). Integrating past $T{=}9$ therefore enters an approximately uniform rectilinear regime: a and κ flatten, $\|v\|$ stays roughly constant, and x drifts indefinitely along a fixed direction. That continuation leaves the domain that stage 4 and the classification head were trained on, and predictions collapse, as in the overthinking of recurrent networks run past their training depth (Bansal et al., 2022). If instead the block at $T{=}9$ were the identity ($v{=}0$), the state would sit still and one could in principle integrate indefinitely without that drift, as in deep equilibrium models that solve directly for such a fixed point (Bai et al., 2019); the trained shared field does not realise that fixed point.

The untied ConvNeXt, Swin and ResNet behave differently: with $R{=}100$ and Euler step 1 (blocks never trained for reuse) the dynamics resemble a coarse version of the $R{=}100$, $\Delta t{=}0.01$ refinement—larger magnitude, still tied to block boundaries rather than a single coasting field.

3.4. Are over-parametrized networks undoing what the previous layers have done?

Section 3.1 showed that one shared stage-3 block nearly matches nine independent ones (-0.61 top-1 points, 33% fewer parameters). A natural explanation is that the untied stack is over-parametrised along depth: consecutive blocks need not push the state in a coherent direction, and part of what block $k{+}1$ does can cancel what block k just did. Residual stages have been read as iterative refinement of a single representation (Greff et al., 2017; Jastrzębski et al., 2018); we ask how much of that refinement is net progress. We test that claim directly on feature trajectories, using the same 500 ImageNet validation images (one per class) as in Section 3.3, stage 3 only, with the massive channel removed (Section 3.3.1).

Setup. For each image we integrate the stage from $t{=}0$ to $t{=}9$ at native resolution ($R{=}1$) and under Euler refinement ($R{=}100$, $\Delta t{=}0.01$), and reduce every spatial feature coordinate whose net displacement $|\Delta_i| = |x_i(9) - x_i(0)|$ is above the per-image median. We partition the trajectory into the nine unit intervals $[k, k{+}1]$ (native blocks in ConvNeXt-T; the same time bins in the weight-tied model) and write

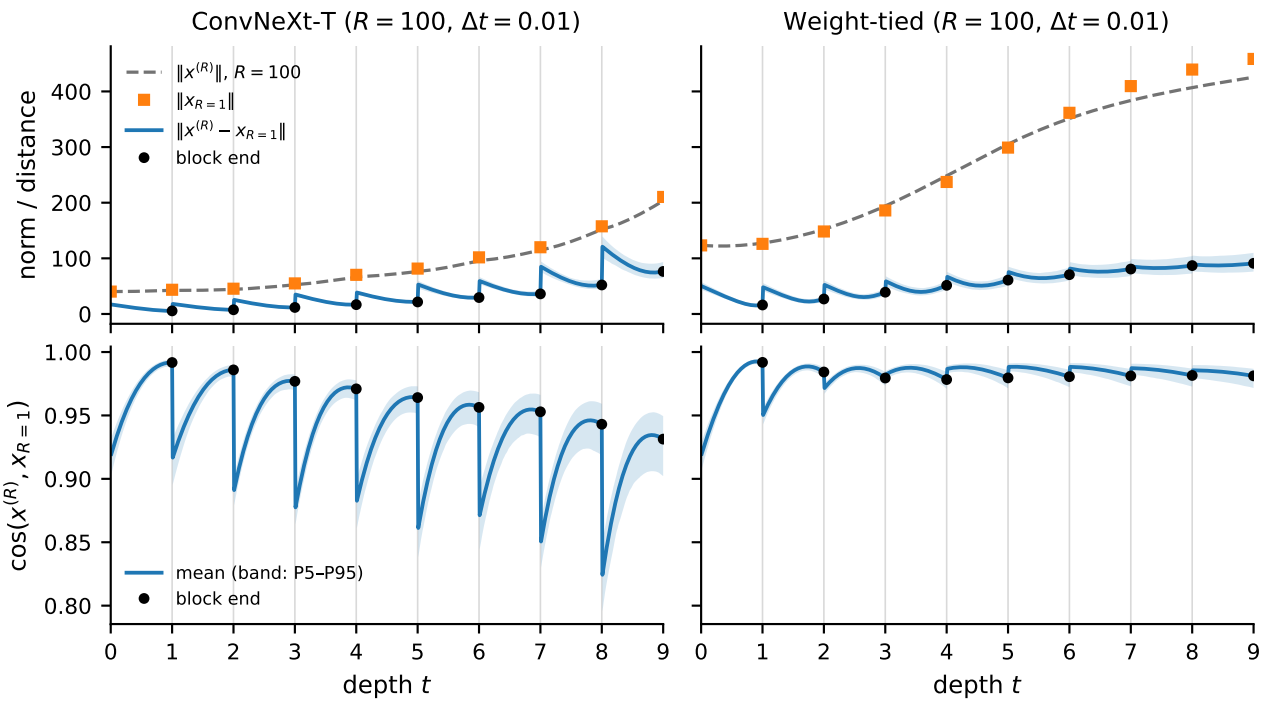


Figure 7. Refined vs. native (R=1) stage-3 path for the standard (left) and weight-tied (right) ConvNeXt-T. **Top:** $\|x^{(R)} - x_{R=1}\|$ (solid) with both state norms for scale (dashed: refined; squares: native). **Bottom:** $\cos(x^{(R)}, x_{R=1})$. Mean and P5–P95 over the same 500 images; massive channel ignored; dots mark the end of each native step. The switched stack accumulates angle at every block; the shared field saturates.

Table 2. Pooled stage-3 trajectory statistics over 500 ImageNet images (massive channel ignored; coordinates with $|\Delta_i|$ above the per-image median). Weight-tied vs. standard ConvNeXt-T at native (R=1) and refined (R=100) resolution. Mean ± std across images.

Model	R	straightness	wasted motion	monotone share	$\cos(\Delta x_k, \Delta x_{k+1})$	reversal rate	undo fraction	zig-zag share	sign flips at $t \in \mathbb{N}$	median t_{50}
Weight-tied ConvNeXt	100	0.78±0.02	0.19±0.01	0.73±0.02	0.92±0.01	0.11±0.01	0.02±0.00	0.14±0.02	0.00±0.00	4.09±0.16
ConvNeXt-T	100	0.50±0.01	0.48±0.01	0.21±0.01	0.17±0.01	0.45±0.00	0.25±0.00	0.90±0.00	0.45±0.00	7.21±0.15
Weight-tied ConvNeXt	1	0.72±0.02	0.25±0.02	0.63±0.03	0.77±0.03	0.18±0.01	0.05±0.01	0.34±0.03	0.18±0.01	5.00±0.09
ConvNeXt-T	1	0.50±0.01	0.48±0.01	0.22±0.01	0.16±0.01	0.45±0.00	0.25±0.00	0.90±0.00	0.45±0.00	7.82±0.38

$\mathrm{d}x_k$ for the displacement of a coordinate over interval k. The scalars we report are means ± std over the 500 images (Table 2):

- *straightness*: $\|\sum_k \mathrm{d}x_k\| \,/\, \sum_k \|\mathrm{d}x_k\|$ in feature space (path length vs. net displacement);
- *wasted motion*: $1 - \sum_i |\mathrm{net}_i| / \sum_{k,i} |\mathrm{d}x_{k,i}|$, the fraction of per-coordinate travel that cancels before t=9;
- $\cos(\mathrm{d}x_k, \mathrm{d}x_{k+1})$: mean cosine between consecutive block displacements;
- *undo fraction*: when $\mathrm{sign}(\mathrm{d}x_k)$ flips at the next block, the cancelled mass $\min(|\mathrm{d}x_k|, |\mathrm{d}x_{k+1}|)$ normalised by $\sum |\mathrm{d}x_k|$;
- *zig-zag share*: mass-weighted share of coordinates that reverse at least twice across the nine blocks;
- *sign flips at $t \in \mathbb{N}$*: fraction of consecutive steps whose sign changes exactly at integer depths (block boundaries).

Results. Table 2 and Figure 8 give a consistent picture. Under refinement (R=100), the weight-tied model is nearly monotone along depth: straightness 0.78, wasted motion 0.19, consecutive-block cosine 0.92, undo fraction 0.02, and zig-zag share 0.14. Sign changes at integer times are essentially absent (0.00). ConvNeXt-T, with nine independent blocks, sits at the opposite extreme: straightness 0.50, wasted motion 0.48 (almost half of the travel cancels), cosine 0.17, undo fraction 0.25 ($\approx 12\times$ the shared value), and zig-zag share 0.90. Nearly half of the sign flips (0.45) fall exactly on block boundaries—the discontinuities of the switched field in Section 3.2.2, not an artefact of a coarse step. Native resolution (R=1) barely changes

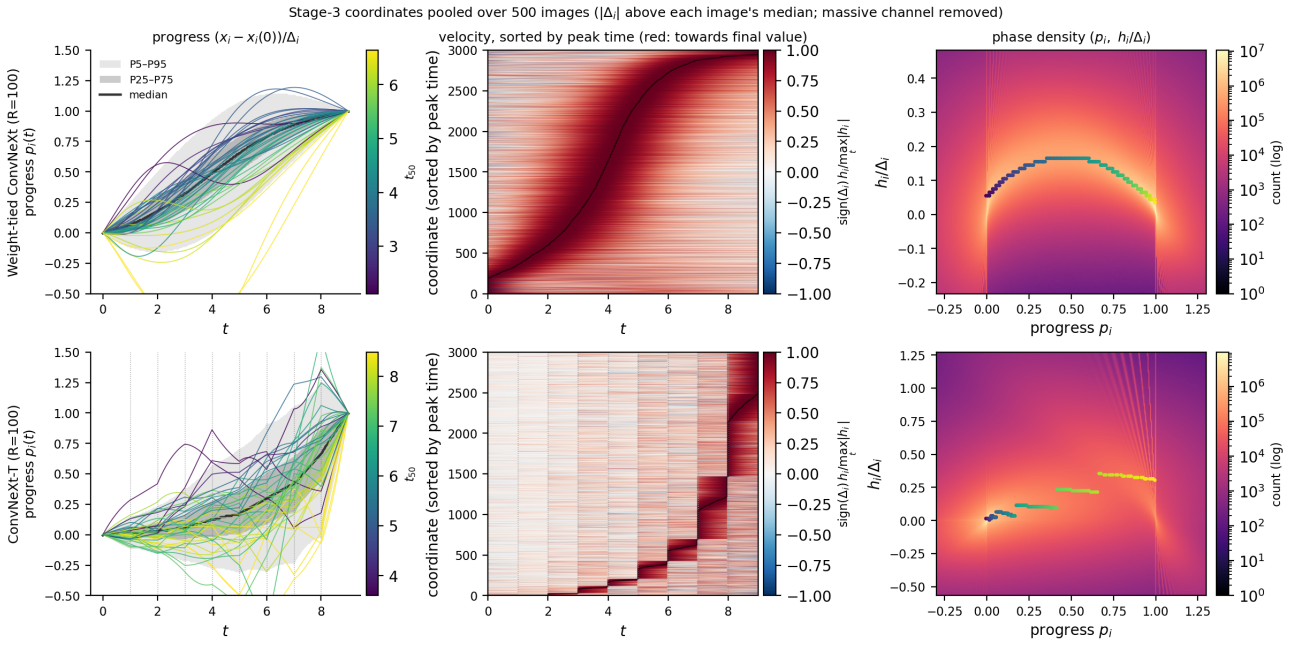


Figure 8. Stage-3 coordinates pooled over 500 images (R=100; massive channel removed; $|\Delta_i|$ above the per-image median). **Top:** weight-tied ConvNeXt; **bottom:** ConvNeXt-T. **Left:** progress $p_i(t) = (x_i(t) - x_i(0))/\Delta_i$ (median and P5–P95; thin lines: random coordinates). **Middle:** velocity toward the final value, coordinates sorted by peak time (red: toward the end state; blue: away). **Right:** phase density $(p_i,\ h_i/\Delta_i)$. The shared field advances mostly monotonically; the switched stack reverses and rewrites features between blocks.

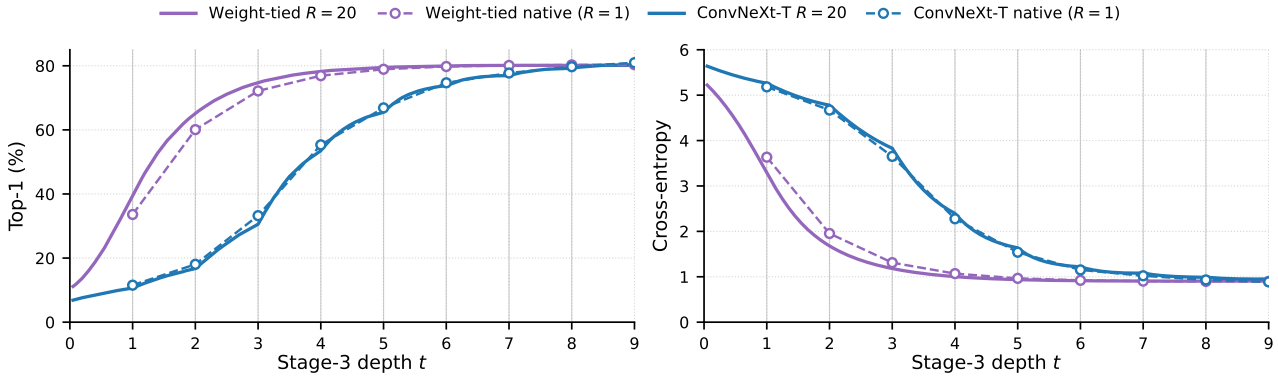


Figure 9. ImageNet top-1 (left) and cross-entropy (right) of each model's trained stage 4+head on stage 3 prefixes (full validation). After each prefix we skip the rest of stage 3 and decode. **Solid:** refined path (R=20, Δt=0.05). **Open markers:** native discrete prefixes (R=1) at block ends. Weight-tied features become usable early and smoothly; ConvNeXt-T postpones most of the gain until late in the stage.

the ConvNeXt-T numbers; refining only makes the shared field look cleaner (undo $0.05 \to 0.02$), which matches the bounded refinement error of Section 3.3.4.

Functional progress along depth. Geometry shows wasted motion; a complementary probe asks when the stage 3 state becomes usable by the rest of the network. After a prefix of stage 3 we skip the remainder of the stage and classify with the model's own trained downsample, stage 4 and head (Figure 9; full ImageNet validation, 50,000 images). This is a frozen-decoder variant of intermediate-layer probes (Alain & Bengio, 2017) and of the logit and tuned lens in language models (Belrose et al., 2023): no probe is trained, so the score measures how close each prefix is to

the state the decoder expects. Solid curves integrate the prefix at $R{=}20$ ($\Delta t{=}0.05$) and decode after every micro-step; connected markers are the native discrete prefixes ($R{=}1$, one evaluation at each block end). Under weight tying the top-1 rises early and smoothly: at $t{=}1, 3, 5$ the refined path already scores 39.5%, 74.7% and 79.5%, against a final 80.1% (native prefixes: 33.6%, 72.2%, 78.9%, final 80.3%). The cross-entropy falls the same way ($3.29 \to 1.18 \to 0.94$ refined, $3.63 \to 1.31 \to 0.97$ native; floor 0.90). On untied ConvNeXt-T the same frozen decoder stays weak for longer (10.7%, 30.6%, 65.5% refined; 11.5%, 33.2%, 66.9% native; losses 5.26, 3.83, 1.64 refined and 5.18, 3.65, 1.54 native) and only approaches the final 80.3%–81.0% in the last few blocks. The tied curve is a single sigmoid; the untied curve is piecewise, with most of the gain between integers—the same block-wise rewrites as in Figure 6. Native and refined tracks stay close, so the contrast is not an artefact of the solver: if anything, refining the shared field *helps* early prefixes ($+6$ pp at $t{=}1$), whereas refining the untied stack sits slightly *below* the native block ends the head was trained on. Mid-stage accuracy is not a claim that early features lack information—the head was trained on x_T—but it does show that the tied path enters the decoder's operating region steadily, whereas the untied path aligns with that region late, consistent with a transport that rewrites features before committing.

Reading. Untied stage 3 does not implement nine steps of one transport: consecutive blocks often point in weakly related or opposing directions and cancel a large fraction of each other's work. The weight-tied network cannot cancel itself in that way—a single autonomous field has no "next block" with different weights—and the geometry shows almost no undoing. Figure 9 makes the same contrast in function space: tied depth builds decoder-ready features gradually, while untied depth postpones most of the usable progress. That is why sharing the nine blocks costs less than one top-1 point (Section 3.1): much of the untied capacity along depth is spent rewriting features rather than accumulating a coherent displacement toward x_T. The residual accuracy gap to the baseline is then more plausibly attributed to the small, non-cancellable specialization that independent θ_k still provide, not to a fundamentally different dynamical mechanism.

4. Discussion

The weight-tied stage 3 was introduced as a control: the simplest residual stage whose refinement is a genuine autonomous ODE. It ended up as the other half of the argument. Read side by side, the two models show that the same residual algebra $x + f(x)$ supports two different transports, and that the difference explains most of what nine independent blocks buy. Table 3 collects the numbers that carry this reading.

Why one shared block is enough. One shared residual map, applied nine times, carries almost all of the accuracy of an untied ConvNeXt-T stage 3 (-0.61 top-1 with 33% fewer parameters), and sharing every stage still reaches 79.01% with 70% fewer. We read this as a consequence of how the shared block is trained. Because it is applied at every depth, it must act sensibly on every state it visits, so it cannot defer work to a later block that does not exist. The prefix probe shows this directly (Figure 9): decoding with the frozen stage 4 and head, the tied path already reaches 39.5% at $t{=}1$ and 74.7% at $t{=}3$, against 10.7% and 30.6% for the untied stack; half of the net displacement is done by $t{\approx}4.1$ instead of 7.2 (Table 2). The kinematics agree (Figure 6). The untied stack accelerates toward the end ($\|v\|$ grows from $\approx$17 to $\approx$112 across blocks), whereas the shared field makes its largest step in mid-stage and spends the last third braking and turning, i.e. refining. This is close to the iterative-refinement reading of residual stages (Greff et al., 2017; Jastrzębski et al., 2018) and to the observation that a weight-shared ResNet behaves as a recurrent network at little cost (Liao & Poggio, 2016); our contribution is to show what that refinement looks like as a trajectory.

The untied stack spends depth undoing itself. The nine independent blocks do not implement nine steps of one transport. Almost half of their per-coordinate travel cancels before $t{=}9$ (wasted motion 0.48, undo 0.25), and consecutive block displacements are nearly orthogonal (cosine 0.17); the shared field barely cancels (undo 0.02, cosine 0.92). This redundancy is already present in the native network: the $R{=}1$ and $R{=}100$ statistics of ConvNeXt-T are identical. What refinement adds is *where* it happens. At $R{=}100$, 45% of all sign flips fall on the eight block boundaries, which are under 1% of the transitions, a concentration of about fifty times. Within a block the path is a shallow arc that turns $30°$–$40°$; at the integers it turns by $\approx 80°$ as the next block replaces the field (Section 3.3.3). The redundancy is therefore a property of switching the field, not of the integration inside each block.

A kinematic reading of the stage. These trajectories argue against reading a residual stage as a sequence of new representations—a unit that detects "a wheel" in one block and "a car" in the next. Within a stage the representation is transported, not replaced: the same block can be applied a hundred times at $\Delta t{=}0.01$ and the decoder still reads the result. A more useful picture follows from the manifold hypothesis (Bengio et al., 2013): natural images lie near a low-dimensional manifold, and the stage deforms its image under the encoder into an output manifold where classes are separable, in line with the untangling view of object recognition (DiCarlo & Cox, 2007) and with geometric and

Table 3. Headline measurements on ImageNet-1k. Top-1 and ODE limit from the conserved-horizon sweep of Table 1 (ODE: RK4 at the finest step). Straightness and undo: stage-3 trajectories at R=100 (Table 2). Params and the all-stages run are from Section 3.1. Swin-T and the ResNets are the standard pretrained checkpoints; we did not retrain them.

Model	Params	R=1	ODE	Gap	Str.	Undo
ConvNeXt-T	28.59M	80.95	80.23	−0.72	0.50	0.25
Weight-tied S3	18.97M	80.34	80.12	−0.22	0.78	0.02
All-stages tied	8.68M	79.01	—	—	—	—
Swin-T	—	81.08	80.32	−0.76	—	—
ResNet-50	—	80.34	79.43	−0.91	—	—
ResNet-101	—	81.67	81.05	−0.62	—	—

topological analyses of deep representations (Brahma et al., 2016; Hauser & Ray, 2017; Naitzat et al., 2020; Cohen et al., 2020). In the weight-tied model that deformation is smooth: the state searches for a heading, expands along it, and then refines (Section 3.3.2). In the untied stack it proceeds in jolts: each block bends the state locally, stops, and the next one pushes elsewhere, with increasing speed. Since the same field acts on every input, the 500 trajectories we integrate are a finite sample of how the whole manifold is transported. What we do not measure are properties of the manifold itself, such as its intrinsic dimension or the separability of classes along t; a sample of one image per class is too sparse for that, and we leave it to future work.

Refined and native paths are displaced but equivalent. Every architecture we tested converges to a single continuous limit once the horizon T is held fixed, and that limit keeps the function (gaps 0.2–0.9 points). The native unit-step path is not that limit, and the gap is not small in geometric terms. On ConvNeXt-T the refined endpoint is 21° away from the native state, a relative distance of 0.36; the whole stage rotates the state by only ≈78° from x_0 to x_9. On the weight-tied model the drift saturates at 11° (relative distance 0.20; Section 3.3.4). Yet top-1 moves by 0.7 and 0.2 points. The decoder is thus nearly invariant to large displacements along the directions that refinement explores, which is consistent with a light adaptation of stage 4 and the head restoring the baseline at R=3 (80.93%). What the decoder does depend on is the integration time: breaking the horizon (R repeats at Δt=1) collapses every network to chance, and integrating the shared field past T drifts out of the decoder's domain (Section 3.3.5). A trained stage is therefore a sum of one-step Euler solves, each replaceable by a finer integral of the *same* block, but not a Neural ODE whose parameters vary smoothly with depth: bilinear interpolation of consecutive θ_k costs 6–23 points (Appendix F). A random residual field also has a continuous limit (Appendix D); what training adds is that the limit remains a classifier.

Refinement as a probe. Refining at a fixed horizon requires no retraining and turns any residual stage into a trajectory sampled as finely as needed. The weight-tied model is the clearest case, because its refined path is continuous over the whole stage, but the same procedure separates within-block motion from block switches in untied networks. Nothing in the probe is specific to ConvNeXt. We already apply the refinement to Swin-T and ResNet-50/101 (Section 3.2, Appendices B, D and G), whose limits behave like ConvNeXt's, although the kinematic analysis of Section 3.3 is reported for ConvNeXt only. The velocity decomposition, curvature, undo statistics and prefix decoding apply to any residual stream. In language models the prefix probe is the logit or tuned lens (Belrose et al., 2023), trajectory geometry across layers is starting to be measured (Pandey et al., 2026), and looped or universal transformers (Dehghani et al., 2019; Giannou et al., 2023) are the weight-tied counterpart of our shared stage.

Limitations. All results are on ImageNet-1k classification, and the trained comparison between tied and untied stages uses one architecture, ConvNeXt-T. The trajectory metrics exclude the massive ConvNeXt channel and the Swin corner tokens (Section 3.3.1); they describe the remaining state, and the role of the massive channel as a depth signal remains a conjecture. The prefix probe uses a decoder trained only on x_T, so low mid-stage accuracy means the state has not yet entered the decoder's operating region, not that it lacks information. Finally, we track a sample of the manifold rather than its geometry; pairwise distances or class separability along t, on many images per class, would make that reading quantitative.

Toward more expressivity per parameter. The leftover 0.6 points are what independent θ_k still specialise, but the untied stack pays for them with a large share of cancelled motion. An architecture that keeps every parameter pushing toward the same goal would combine a mostly autonomous field with a controlled degree of non-autonomy. Two directions follow from our results: small per-block corrections on top of a shared block, regularised so that the field stays close to autonomous, and cycles of a few shared blocks instead of one. Related ideas exist for convolutional networks (Savarese & Maire, 2019) and transformers, through cyclic layer sharing (Takase & Kiyono, 2023) and shared layers with layer-wise low-rank updates (Bae et al., 2025). Designing and training such an architecture is beyond the scope of this work.

5. Conclusion

Residual blocks can be refined indefinitely at inference: as long as the integration horizon T is kept, ConvNeXt, Swin and ResNet stages converge to a continuous limit that loses

less than one top-1 point, whereas breaking T collapses every network. Refinement turns a residual stage into a trajectory, and that trajectory separates two regimes behind the same algebra $x + f(x)$. A single shared block, reused over the whole stage, is an autonomous transport: it searches for a heading, expands along it and refines, and its features become usable by the decoder early. Nine independent blocks form a switched transport: shallow arcs joined by sharp turns at every block boundary, where almost half of the motion is undone. This redundancy is why one shared block recovers almost all of the accuracy of the untied stage (-0.6 points with 33% fewer parameters). The same probe applies to any residual stream, and suggests a design goal: fields that stay mostly autonomous, so that every parameter pushes toward the same target.

References

Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations (ICLR)*, 2023.

Alain, G. and Bengio, Y. Understanding intermediate layers using linear classifier probes. In *International Conference on Learning Representations (ICLR), Workshop Track*, 2017. arXiv:1610.01644.

Ansuini, A., Laio, A., Macke, J. H., and Zoccolan, D. Intrinsic dimension of data representations in deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Bae, S., Fisch, A., Harutyunyan, H., Ji, Z., Kim, S., and Schuster, T. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA. In *International Conference on Learning Representations (ICLR)*, 2025.

Bai, S., Kolter, J. Z., and Koltun, V. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

Bansal, A., Schwarzschild, A., Borgnia, E., Emam, Z., Huang, F., Goldblum, M., and Goldstein, T. End-to-end algorithm synthesis with recurrent networks: Extrapolation without overthinking. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Belrose, N., Furman, Z., Smith, L., Halawi, D., Ostrovsky, I., McKinney, L., Biderman, S., and Steinhardt, J. Eliciting latent predictions from transformers with the tuned lens, 2023. arXiv preprint.

Bengio, Y., Simard, P., and Frasconi, P. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. doi: 10.1109/72.279181.

Bengio, Y., Courville, A., and Vincent, P. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35 (8):1798–1828, 2013.

Boulch, A. ShaResNet: reducing residual network parameter number by sharing weights, 2017. URL https://arxiv.org/abs/1702.08782.

Brahma, P. P., Wu, D., and She, Y. Why deep learning works: A manifold disentanglement perspective. *IEEE Transactions on Neural Networks and Learning Systems*, 27(10), 2016.

Chang, B., Meng, L., Haber, E., Tung, F., and Begert, D. Multi-level residual networks from dynamical systems view. In *International Conference on Learning Representations (ICLR)*, 2018.

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

Cohen, A.-S., Cont, R., Rossier, A., and Xu, R. Scaling properties of deep residual networks. In *International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pp. 2039–2048, 2021.

Cohen, U., Chung, S., Lee, D. D., and Sompolinsky, H. Separability and geometry of object manifolds in deep neural networks. *Nature Communications*, 11:746, 2020.

Darcet, T., Oquab, M., Mairal, J., and Bojanowski, P. Vision transformers need registers. In *International Conference on Learning Representations*, 2024.

Dasgupta, S. and Gupta, A. An elementary proof of a theorem of Johnson and Lindenstrauss. *Random Structures & Algorithms*, 22(1):60–65, 2003.

Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, Ł. Universal transformers. In *International Conference on Learning Representations (ICLR)*, 2019.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 248–255, 2009.

DiCarlo, J. J. and Cox, D. D. Untangling invariant object recognition. *Trends in Cognitive Sciences*, 11(8):333–341, 2007.

Dupont, E., Doucet, A., and Teh, Y. W. Augmented neural ODEs. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

E, W. A proposal on machine learning via dynamical systems. *Communications in Mathematics and Statistics*, 5 (1):1–11, 2017.

Entezari, R., Sedghi, H., Saukh, O., and Neyshabur, B. The role of permutation invariance in linear mode connectivity of neural networks. In *International Conference on Learning Representations (ICLR)*, 2022.

Fernando, J. and Guitchounts, G. Transformer dynamics: A neuroscientific approach to interpretability of large language models, 2025.

Gan, C., Tu, Y., Chen, X., Chen, T., Li, Y., Harandi, M., and Lin, W. Unleashing diffusion transformers for visual correspondence by modulating massive activations. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

Geiping, J., McLeish, S., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A., and Goldstein, T. Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Giannou, A., Rajput, S., yong Sohn, J., Lee, K., Lee, J. D., and Papailiopoulos, D. Looped transformers as programmable computers. In *International Conference on Machine Learning (ICML)*, 2023.

Glorot, X. and Bengio, Y. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 249–256, 2010.

Greff, K., Srivastava, R. K., and Schmidhuber, J. Highway and residual networks learn unrolled iterative estimation. In *International Conference on Learning Representations (ICLR)*, 2017.

Gromov, A., Tirumala, K., Shapourian, H., Glorioso, P., and Roberts, D. A. The unreasonable ineffectiveness of the deeper layers. In *International Conference on Learning Representations (ICLR)*, 2025.

Haber, E. and Ruthotto, L. Stable architectures for deep neural networks. *Inverse Problems*, 34(1):014004, 2017.

Hairer, E., Nørsett, S. P., and Wanner, G. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, 2 edition, 1993.

Halko, N., Martinsson, P.-G., and Tropp, J. A. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

Hauser, M. and Ray, A. Principles of Riemannian geometry in neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.

Hénaff, O. J., Goris, R. L. T., and Simoncelli, E. P. Perceptual straightening of natural videos. *Nature Neuroscience*, 22(6):984–991, 2019.

Hosseini, E. A. and Fedorenko, E. Large language models implicitly learn to straighten neural sentence trajectories to construct a predictive representation of natural language. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Huang, G., Sun, Y., Liu, Z., Sedra, D., and Weinberger, K. Q. Deep networks with stochastic depth. In *European Conference on Computer Vision (ECCV)*, 2016.

Ioffe, S. and Szegedy, C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, 2015.

Jastrzębski, S., Arpit, D., Ballas, N., Verma, V., Che, T., and Bengio, Y. Residual connections encourage iterative inference. In *International Conference on Learning Representations (ICLR)*, 2018.

Johnson, W. B. and Lindenstrauss, J. Extensions of Lipschitz mappings into a Hilbert space. *Contemporary Mathematics*, 26:189–206, 1984.

Lad, V., Lee, J. H., Gurnee, W., and Tegmark, M. The remarkable robustness of LLMs: Stages of inference? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2406.19384.

Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., and Soricut, R. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations (ICLR)*, 2020.

Li, H., Xu, Z., Taylor, G., Studer, C., and Goldstein, T. Visualizing the loss landscape of neural nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

Liao, Q. and Poggio, T. Bridging the gaps between residual learning, recurrent neural networks and visual cortex, 2016.

Liberzon, D. *Switching in Systems and Control*. Birkhäuser, Boston, 2003.

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., and Guo, B. Swin Transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., and Xie, S. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

Lu, Y., Zhong, A., Li, Q., and Dong, B. Beyond finite layer neural networks: Bridging deep architectures and numerical differential equations. In *International Conference on Machine Learning (ICML)*, 2018.

Marion, P., Wu, Y.-H., Sander, M. E., and Biau, G. Implicit regularization of deep residual networks towards neural ODEs. In *International Conference on Learning Representations (ICLR)*, 2024.

Naitzat, G., Zhitnikov, A., and Lim, L.-H. Topology of deep neural networks. *Journal of Machine Learning Research*, 21(184):1–40, 2020.

Pandey, V., Singh, G., and Mahdid, Y. Trajectory geometry of transformer representations across layers, 2026. URL https://arxiv.org/abs/2606.09287.

Queiruga, A. F., Erichson, N. B., Taylor, D., and Mahoney, M. W. Continuous-in-depth neural networks, 2020. URL https://arxiv.org/abs/2008.02389.

Sander, M. E., Ablin, P., and Peyré, G. Do residual neural networks discretize neural ordinary differential equations? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 2022.

Savarese, P. and Maire, M. Learning implicitly recurrent CNNs through parameter sharing. In *International Conference on Learning Representations (ICLR)*, 2019.

Simonyan, K. and Zisserman, A. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

Strogatz, S. H. *Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering*. Westview Press, 2 edition, 2015.

Sun, M., Chen, X., Kolter, J. Z., and Liu, Z. Massive activations in large language models. In *Conference on Language Modeling (COLM)*, 2024.

Sun, Q., Pickett, M., Nain, A. K., and Jones, L. Transformer layers as painters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 25219–25227, 2025.

Sun, S., Canziani, A., LeCun, Y., and Zhu, J. Anatomy of massive activations and attention sinks. In Zhang, T., Dudik, M., Jaggi, M., Agarwal, A., Li, S., Schuurmans, D., Zhu, J., Berkenkamp, F., Dong, H., and Bietti, A. (eds.), *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pp. 116443–116463. PMLR, 06–11 Jul 2026. URL https://proceedings.mlr.press/v306/sun26d.html.

Takase, S. and Kiyono, S. Lessons on parameter sharing across layers in transformers. In *Proceedings of the Fourth Workshop on Simple and Efficient Natural Language Processing (SustaiNLP)*, pp. 78–90, 2023. doi: 10.18653/v1/2023.sustainlp-1.5.

Valeriani, L., Doimo, D., Cuturello, F., Laio, A., Ansuini, A., and Cazzaniga, A. The geometry of hidden representations of large transformer models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Veit, A., Wilber, M., and Belongie, S. Residual networks behave like ensembles of relatively shallow networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29, 2016.

A. Training recipe

We train on ImageNet-1k (Deng et al., 2009) at 224×224. Table 4 lists the full hyper-parameter table, next to the official ConvNeXt-T recipe (Liu et al., 2022). The four models we train from scratch share the same augmentation, regularisation and optimiser; they differ only in peak learning rate, warmup length, minimum learning rate and stochastic depth, as marked in the table. The official ConvNeXt-T uses batch size 4096 and base learning rate 4×10^{-3}. Linear scaling to our batch of 1024 gives 1×10^{-3}, which is the peak we use for the weight-tied and all-stages runs. The untied ConvNeXt-T (and its drop-path-free sibling) use half of that, 5×10^{-4}, with the official 20-epoch warmup.

Weight decay is applied only to convolution and linear weights (tensors of rank ≥ 2). LayerNorm gain and bias, convolution/linear bias and LayerScale γ are decay-free, following the ConvNeXt / DeiT convention. Convolution and linear weights are initialised with truncated normal $\mathcal{N}(0, 0.02^2)$; LayerScale uses γ=10^{-6}. There is no layer-wise learning-rate decay, no gradient clipping, no mixed precision and no exponential moving average of the weights. The learning-rate schedule is a linear warmup (starting at 10^{-6}) followed by cosine decay to the listed minimum, advanced once per optimiser step.

Mixup and CutMix are applied to every training batch (switch probability 0.5), and the loss is soft-target cross-entropy on the mixed labels. Per-image augmentation is the timm ConvNeXt stack: random resized crop, horizontal flip, RandAugment `rand-m9-mstd0.5`, color jitter 0.4 and random erasing (probability 0.25, pixel mode). Validation uses the standard centre crop. Stochastic depth, when enabled, is ramped linearly from 0 to the listed peak across the 18 residual blocks.

Stage-4 + head finetune after unrolling. The decoder-adaptation probe of Section 3.2 freezes stem and stages 1–3, unrolls stage 3 at inference resolution (R=100, Δt=0.01, T=9; 900 residual steps on the weight-tied model) and trains only stage 4, the global pool and the classification head. We run 10 epochs at a constant learning rate 5×10^{-6} (no warmup, no cosine decay), with the same optimiser, weight decay, batch size and augmentation as Table 4.

B. Full Midpoint and RK4 refinement tables

We use the same R (repetitions) as in the main text: Δt=$1/R$ so that T=n is conserved (n=9 for ConvNeXt stage 3). Tables 5–8 report plain interpolation for Euler (RK1), Midpoint (RK2) and RK4 on each pretrained model. Table 9 gives the same solvers for the weight-tied ConvNeXt-T at fixed horizon T=9 ($9R$ applications of the shared block). In every case the three integrators converge to the same continuous limit once the step is fine enough; at R=1, Mid and RK4 already sit on that limit.

C. Solver agreement at the stage-3 endpoint

Section 3.2 asks whether a fine Euler step recovers the native R=1 state. A related question is whether two fine discretisations of the *same* residual field agree with each other—i.e. whether a continuous limit exists at all. Table 10 compares stage-3 endpoints x_T on the same N=32 ImageNet validation images (one per random class, seed 0), after zeroing the massive channel used in the main-text metrics. Relative distance is $\|x_a - x_b\|/\|x_b\|$; cosine is $\cos(x_a, x_b)$.

Euler and RK4 (and Euler R=100 vs. R=200) collapse onto the same x_T, while both remain far from the trained one-step path. RK4 reaches that limit already with the nine unit steps of the native schedule; the native Euler path does not. Midpoint at R=1 matches the limit in top-1 (80.23%, Table 5) while its state is still 12% away, a reminder that accuracy is a coarse probe of x_T.

D. Random-initialisation control for the continuous limit

Appendix C shows that the trained stage 3 has a continuous limit. That alone is weak evidence. Every residual field we study (convolutions, linear layers, normalisation, GELU/ReLU, softmax attention) is locally Lipschitz with at most linear growth—attention outputs are convex combinations of the values—so the ODE $\dot{x} = f_i(x)$ has a unique solution on $[0, T]$ and Euler converges to it at first order, whatever the weights (Hairer et al., 1993). We check this empirically on all five architectures and use random fields of controlled strength as a reference scale for the trained ones.

Setup. Every convolution and linear layer is drawn with Xavier-normal initialisation (Glorot & Bengio, 2010) (gain 1, zero bias, seed 0); normalisation layers keep their default parameters, and ConvNeXt LayerScale is set to γ=1 (its default 10^{-6} makes $f \approx 0$). The stage-3 blocks are then re-drawn with Xavier gain $g \in \{0.25, 0.5, 1, 2, 4, 8\}$, so the stage-3 input is identical across gains and only the field strength changes. For ResNet the field is $f_i(x) = \mathrm{block}_i(x) - x$, which includes the ReLU applied after the addition; for Swin one block contains both residual sublayers (attention and MLP), as in the main text. For each network we integrate stage 3 to T=n and compare the endpoint x_T of Euler at $R \in \{1, 10, 100, 1000\}$ and of RK4 at R=1 against RK4 at R=100, on the N=32 images of Appendix C. The field strength is summarised by the mean

Table 4. ImageNet-1k training settings. Official ConvNeXt-T from Liu et al. (2022); the remaining columns are the four models we train. CNX-T: untied baseline. Tied: stage 3 shares one block (D=9). All: one shared block per stage. dp0: untied ConvNeXt-T with stochastic depth off.

	Official	CNX-T	Tied	All	dp0
training epochs	300	300	300	300	300
warmup epochs	20	20	10	10	20
optimizer	AdamW	AdamW	AdamW	AdamW	AdamW
base learning rate	4×10^{-3}	5×10^{-4}	1×10^{-3}	1×10^{-3}	5×10^{-4}
min. learning rate	—	10^{-6}	10^{-7}	10^{-7}	10^{-6}
weight decay	0.05	0.05	0.05	0.05	0.05
optimizer momentum	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)	β=(0.9, 0.999)
batch size	4096	1024	1024	1024	1024
learning rate schedule	cosine	cosine	cosine	cosine	cosine
warmup schedule	linear	linear	linear	linear	linear
layer-wise lr decay	None	None	None	None	None
RandAugment	(9, 0.5)	(9, 0.5)	(9, 0.5)	(9, 0.5)	(9, 0.5)
Mixup α	0.8	0.8	0.8	0.8	0.8
CutMix α	1.0	1.0	1.0	1.0	1.0
random erasing	0.25	0.25	0.25	0.25	0.25
color jitter	0.4	0.4	0.4	0.4	0.4
label smoothing	0.1	0.1	0.1	0.1	0.1
stochastic depth	0.1	0.1	0.1	0.1	0
LayerScale init	10^{-6}	10^{-6}	10^{-6}	10^{-6}	10^{-6}
gradient clip	None	None	None	None	None
exp. moving average	None	None	None	None	None
mixed precision	—	off	off	off	off

Table 5. ConvNeXt-T, plain, top-1 (%).

R	Euler	Mid	RK4
1	80.95	80.23	80.23
2	80.79	80.23	80.23
4	80.53	80.23	80.23
10	80.36	80.23	80.23
32	80.30	80.23	80.23
128	80.23	80.22	80.23

Table 6. Swin-T, plain, top-1 (%).

R	Euler	Mid	RK4
1	81.08	80.29	80.33
2	80.90	80.34	80.31
4	80.65	80.33	80.32
10	80.47	80.32	80.31
32	80.38	80.32	80.32
128	80.34	80.31	80.32

Table 7. ResNet-50, plain, top-1 (%).

R	Euler	Mid	RK4
1	80.34	78.94	79.42
2	80.06	79.41	79.43
4	79.84	79.44	79.43
10	79.61	79.44	79.43
32	79.50	79.43	79.44
128	79.45	79.43	79.43

Table 8. ResNet-101, plain, top-1 (%).

R	Euler	Mid	RK4
1	81.67	80.95	81.06
2	81.58	81.07	81.04
4	81.38	81.06	81.05
10	81.22	81.05	81.05
32	81.12	81.05	81.04
128	81.06	81.04	81.05

relative step of the native path,

$$\rho \;=\; \frac{1}{n}\sum_{i=0}^{n-1}\frac{\|f_i(x_i)\|}{\|x_i\|}, \qquad x_{i+1} = x_i + f_i(x_i), \quad (10)$$

i.e. how far a single block moves the state relative to its size. Because ρ grows with g at a different rate in each architecture (about g^3 for ResNet, whose bottleneck has three unnormalised convolutions), we compare trained and random networks at matched ρ: for each architecture we add one gain $g^\star$ chosen so that ρ matches the trained network. Trained ConvNeXt rows zero the massive channel (Section 3.3.1); ResNet and Swin have no channel of comparable dominance at stage 3 (zeroing the largest one changes

Table 9. Shared ConvNeXt-T, fixed $T{=}9$ ($\Delta t{=}1/R$; $9R$ shared steps), top-1 (%).

R	Steps	Δt	Euler	Mid	RK4
$1/9$	1	9	47.06	0.61	2.70
$2/9$	2	4.5	5.85	10.42	47.98
$3/9$	3	3	37.45	54.47	71.37
$4/9$	4	2.25	67.83	73.57	78.90
$5/9$	5	1.8	77.23	78.70	79.97
$6/9$	6	1.5	79.48	79.85	80.10
$7/9$	7	1.29	80.19	80.12	80.11
$8/9$	8	1.13	80.38	80.21	80.11
1	9	1	80.34	80.19	80.11
$4/3$	12	0.75	80.35	80.14	80.12
2	18	0.5	80.27	80.12	80.12
$32/9$	32	0.28	80.22	80.11	80.12
$128/9$	128	0.07	80.14	80.12	80.12
$1024/9$	1024	0.009	80.12	80.12	80.12
$10000/9$	10000	0.0009	80.12	—	—

Table 10. Pairwise stage-3 endpoint agreement (mean over $N{=}32$; ignore top-1 channel). Horizon $T{=}9$ in every row.

Pair (a vs. b)	rel. ℓ_2	cos	angle
ConvNeXt-T			
Euler $R{=}100$ vs. RK4 $R{=}100$	0.004	1.000	0.25°
Euler $R{=}100$ vs. Euler $R{=}200$	0.002	1.000	0.12°
Euler $R{=}100$ vs. Euler $R{=}1$	0.36	0.93	21°
Mid $R{=}1$ vs. Euler $R{=}100$	0.12	0.993	6.6°
RK4 $R{=}1$ vs. Euler $R{=}100$	0.010	0.9999	0.5°
Weight-tied ConvNeXt-T ($9R$ shared steps)			
Euler $R{=}100$ vs. RK4 $R{=}100$	0.002	1.000	0.1°
Euler $R{=}100$ vs. Euler $R{=}200$	0.001	1.000	0.05°
Euler $R{=}100$ vs. Euler $R{=}1$	0.20	0.98	11°
RK4 $R{=}1$ vs. RK4 $R{=}100$	0.003	1.000	0.16°

every entry by less than $0.6°$), so all rows except those use all channels.

Table 11. Angle between x_T and the RK4 $R{=}100$ reference ($T{=}n$, mean over $N{=}32$). g: Xavier gain of the stage-3 blocks; ρ: mean relative step (10); *: gain matched to the trained ρ. Rows sorted by ρ.

Init	ρ	Euler			RK4
		$R{=}1$	$R{=}10$	$R{=}100$	$R{=}1$
ConvNeXt-T (nine blocks)					
$g{=}0.5$	0.13	1.5°	0.15°	0.02°	0.00°
$g{=}1$	0.41	15.8°	1.6°	0.16°	0.07°
Trained	**0.54**	**21.1°**	**2.4°**	**0.25°**	**0.53°**
$g^\star{=}1.35$	0.56	30.2°	3.4°	0.34°	0.92°
$g{=}2$	0.77	47.0°	6.7°	0.69°	8.4°
Weight-tied ConvNeXt-T (one block)					
$g{=}0.5$	0.12	2.7°	0.27°	0.03°	0.00°
$g^\star{=}1$	0.30	11.9°	1.3°	0.13°	0.05°
Trained	**0.31**	**10.9°**	**1.0°**	**0.10°**	**0.16°**
$g{=}2$	0.63	27.1°	4.3°	0.46°	5.9°
ResNet-50 ($n{=}5$)					
$g{=}0.5$	0.04	0.4°	0.03°	0.00°	0.01°
$g{=}1$	0.34	7.0°	0.66°	0.07°	0.24°
$g^\star{=}1.15$	0.51	12.0°	1.2°	0.12°	0.48°
Trained	**0.52**	**17.0°**	**1.4°**	**0.15°**	**0.65°**
$g{=}2$	2.29	40.6°	7.8°	0.83°	6.6°
ResNet-101 ($n{=}22$)					
$g{=}0.5$	0.05	0.7°	0.05°	0.01°	0.02°
Trained	**0.30**	**15.3°**	**1.4°**	**0.15°**	**0.74°**
$g^\star{=}0.95$	0.30	8.9°	0.83°	0.08°	0.29°
$g{=}1$	0.35	10.3°	0.97°	0.10°	0.36°
$g{=}2$	2.33	39.6°	5.9°	0.63°	6.4°
Swin-T ($n{=}6$)					
$g{=}0.5$	0.15	1.7°	0.17°	0.02°	0.00°
$g{=}1$	0.48	17.2°	1.8°	0.18°	0.10°
Trained	**0.87**	**19.5°**	**4.3°**	**1.02°**	**2.7°**
$g^\star{=}1.8$	0.89	47.6°	7.1°	0.74°	6.6°
$g{=}2$	0.98	52.7°	8.7°	0.91°	10.5°

Results. Table 11 supports two observations.

(i) The limit exists for every architecture. For every row, trained or random, the Euler error falls by roughly $10\times$ per decade of R, the first-order rate (trained Swin-T is slower between $R{=}10$ and $R{=}100$), and Euler at $R{=}1000$ is within $0.1°$ of the reference. The same holds at the larger gains not shown, with two numerical exceptions that confirm the mechanism rather than contradict it. ResNet at $g \geq 4$ overflows in float32: without a normalisation inside the block, $\|f_i(x)\|$ grows with $\|x\|$, so the state grows exponentially ($\rho \approx 18$ per block at $g{=}4$). Swin at $g{=}8$ is still $15.7°$ away at $R{=}1000$: the attention logits are so large that the softmax is nearly a hard selection, the field becomes very stiff, and a finer step is needed. Convergence of the refinement to a single $\hat{x}_T$ is therefore a property of the residual parameterisation, not something learned.

(ii) The native gap is of the order set by ρ, and training moves it in an architecture-dependent direction. Within each architecture the distance from the single-step path ($R{=}1$) to the limit grows monotonically with ρ, and with $\rho \approx 0.3$–0.9 every trained network lies 10–21° away: one Euler step of size 1 is far from tracking the flow. At matched ρ, however, the trained and random networks differ, and not in a consistent direction. Trained Swin-T and ConvNeXt-T are *closer* to their limit than a random field of the same strength (19.5° vs. 47.6° and 21.1° vs. 30.2°), the weight-tied model is level with it (10.9° vs. 11.9°), and both ResNets are *farther* (17.0° vs. 12.0° and 15.3° vs. 8.9°). RK4 at $\Delta t{=}1$ follows the same split. Training thus changes how well a single step approximates the flow, but there is no universal tendency for single-use blocks to be better (or worse) one-step solvers than random ones. Script: `scripts/random_init_convergence.py`.

E. Block permutation, duplication and substitution

F. Bilinear weight interpolation

Continuous-in-depth work treats the weights as a function of depth $\theta(t)$ expanded in a basis (Queiruga et al., 2020). The piecewise-constant basis recovers a ResNet, and multi-level training refines depth by copying adjacent blocks (Chang et al., 2018); smoother bases are a natural alternative. We test the simplest one, linear interpolation between consecutive blocks,

$$\theta(t) \;=\; (1-\alpha)\,\theta_i + \alpha\,\theta_{i+1}, \qquad \alpha = t - i. \tag{11}$$

Replacing the plain schedule $\theta_t = \theta_{\lfloor t \rfloor}$ by (11) hurts every model we tested: at R=128 the drop vs. plain is between 6.6 and 23.3 points (Table 12). Nothing aligns the hidden units of consecutive blocks; as for independently trained networks, averaging weights that are not permutation-aligned need not give a functional intermediate (Entezari et al., 2022; Ainsworth et al., 2023).

Figure 10 shows the ConvNeXt residual-field norm under plain vs. bilinear schedules at R=100. Plain keeps the piecewise-constant staircase of Figure 2 (right); bilinear adds high-frequency oscillation between block boundaries while preserving the same global growth of $\|f\|$. Bands are P5–P95 across 500 validation images (same suite as Figure 2).

Table 12. Bilinear weight interpolation, Euler, top-1 (%).

R	CNX-T	CNX-T†	Swin-T	R-50	R-101
1	80.96	79.27	81.08	80.34	81.67
2	78.94	72.43	78.93	74.95	72.55
4	76.16	63.13	75.42	70.60	62.42
10	74.42	58.75	73.28	68.58	58.73
128	73.66	57.44	72.14	67.77	57.73
Δ vs plain @128	−6.6	−19.6	−8.2	−11.7	−23.3

†ConvNeXt-T trained without drop path.

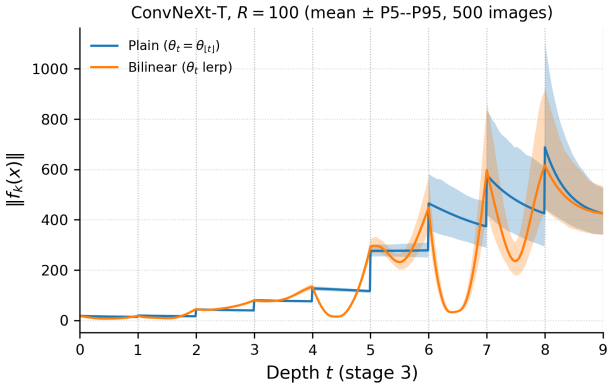


Figure 10. ConvNeXt-T residual norm along stage 3 at R=100: plain vs. bilinear weight interpolation (mean and P5–P95 over 500 images).

G. Massive activations in stage 3

Section 3.3.1 summarises the outliers found in stage 3 of ConvNeXt and Swin. Here we detail how they are identified, how we ablate the ConvNeXt channel and what their kinematics look like. Throughout, we use "massive activation" loosely for channels or tokens that exceed all others by more than an order of magnitude. On the shared and untied ConvNeXt the stage-3 exit also meets the numerical criterion of Sun et al. (2024) (magnitude above 100 and about 10^3 times the median), although here the dominant channel is large at every position rather than at a few tokens, a pattern closer to the uniformly large feature dimension that Sun et al. (2024) report for MAE ViTs and do not count as massive: at every spatial position $\max_c |x_c| > 100$, and the median over positions of $\max_c |x_c| / \mathrm{median}_c |x_c|$ is about $1.1{\times}10^3$ (shared, N=500) and $1.3{\times}10^3$ (untied, N=500). In Swin the corner tokens fail that channel-wise test (ratios of about 16), so the label there refers only to their outlying L2 norms.

Detection. On the shared ConvNeXt we integrate the native stage-3 schedule (D=9, Δt=1, Euler) on a single ImageNet validation image and rank the 384 channels by the Frobenius norm $\|x_c\|_F$ of the stage-3 state just before LayerNorm and downsampling. Channel 236 dominates by more than an order of magnitude ($\|x_{236}\|_F \approx 3.6{\times}10^3$ vs. $\approx 1.4{\times}10^2$ for the runner-up; Figure 11). The same index remains the top channel under the refined schedule (R=10, D=90, Δt=0.1), so the outlier is not an artefact of the unit-step discretisation. An analogous ranking on the untied ConvNeXt-T likewise isolates one channel of comparable relative size; ResNet stage 3 has no channel of this dominance.

Zeroing protocol. To test functional importance we re-run the full ImageNet validation pass while forcing the selected channel to zero after *every* residual step of stage 3 (stem and stages 1–2, 4 are untouched). We compare three conditions under both the native (R=1) and refined (R=10) shared schedules, always at the trained horizon T=9: (i) baseline, no channel forced; (ii) outlier, channel 236 zeroed; (iii) random controls, each of 20 channels drawn uniformly from $\{0,\dots,383\} \setminus \{236\}$ (seed 0) zeroed independently. Table 13 reports top-1 and cross-entropy; Figure 12 shows the corresponding Δloss histogram. Zeroing the outlier costs about 12.5 pp of top-1 and raises the loss by ≈ 0.69, while every random channel stays within 0.3 pp and $\Delta\text{loss} \le 0.02$. The drop is essentially unchanged from R=1 to R=10, so

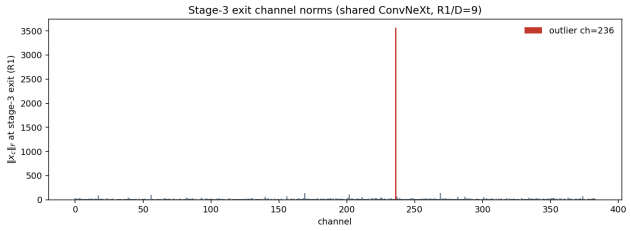


Figure 11. Stage-3 exit channel norms on the shared ConvNeXt (R=1, D=9). Channel 236 (red) is the massive activation used in the ablations below.

the channel remains critical under refinement.

These numbers show that the trained network relies on this channel, but not that a massive channel is necessary: zeroing it also removes its contribution to the LayerNorm statistics at every spatial position and therefore rescales all other channels.

Phase portraits. Figure 13 plots, for every stage-3 channel, the trajectory of spatial means $(\mathrm{mean}_{H,W}(x_d), \mathrm{mean}_{H,W}(h_d))$ over T=9 at Δt=0.01 (N=500 validation images). In the shared model the outlier traces a smooth arc: the residual mean $\mathrm{mean}(h)$ rises then falls while the state mean $\mathrm{mean}(x)$ keeps growing, matching the early acceleration and late velocity drop reported in the main text. In the untied ConvNeXt-T the dominant channel advances in a block-wise sawtooth and moves away from the origin, its magnitude growing with negative sign: its velocity decays within most blocks and jumps at every block boundary. In both models the spatial means of the remaining 383 channels stay close to the origin. This dominance is why we exclude the outlier channel when computing the trajectory-geometry metrics of the main text; the forward pass itself is left unchanged.

Swin. In Swin-T the outliers are concentrated in tokens rather than channels, so means hide them: averaging over channels cancels coordinates of opposite sign, and averaging over positions dilutes four tokens among 196. We therefore plot, for each stage-3 position, the norm over channels of the state $\|x_d\|_C$ against that of the residual $\|h_d\|_C$ (Figure 14, left). The four corner tokens end the stage with norms of about 500 to 600, whereas the median norm over the remaining 192 positions is 23. Most of this growth happens in the last two of the six blocks, where the norm increases by almost the full integral of $\|h_d\|_C$. Since the per-step change of the norm is at most $\Delta t\,\|h_d\|_C$, with equality only when h_d is parallel to x_d, these blocks amplify the corner tokens along their own direction, and the tokens are still growing when the stage ends. Averaged over 500 images, the corner tokens still show large values in the same few

Table 13. Shared ConvNeXt-T ImageNet validation when zeroing one stage-3 channel after every residual step. Random: mean and range over 20 channels.

R	condition	top-1 (%)	loss	Δloss
1	baseline	80.34	0.896	—
1	zero ch 236	67.81	1.595	+0.699
1	zero random (n=20)	80.25 [80.05, 80.38]	—	$\leq$ 0.018
10	baseline	80.14	0.904	—
10	zero ch 236	67.59	1.590	+0.686
10	zero random (n=20)	80.11 [79.85, 80.18]	—	$\leq$ 0.019

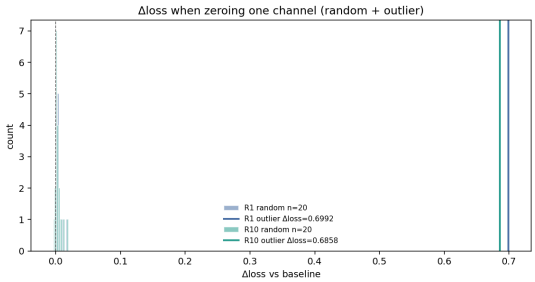


Figure 12. Δloss vs. baseline when zeroing one stage-3 channel on the shared ConvNeXt. Random channels (n=20 per R) cluster near zero; the outlier (vertical lines) sits near Δloss $\approx$ 0.7 for both R=1 and R=10.

channels with the same signs (Figure 15), which suggests a mechanism largely independent of the input. Averaging over positions instead (Figure 14, right), one channel grows above the rest in a block-wise sawtooth similar to that of the untied ConvNeXt, but its final spatial mean is only about three times that of the next largest channel.

Open questions. Two tests would clarify the role of the ConvNeXt channel. Following the set-to-mean intervention of Sun et al. (2024), replacing it by its mean over images at the same step, instead of zero, would also approximately preserve the LayerNorm statistics and thus separate the effect of the channel from that of the renormalisation. Tracking the ratio between the channel and the RMS of the remaining channels along depth separates the two readings of Section 3.3.1: as an implicit parameter the ratio should stay roughly constant, whereas as a depth signal it should vary systematically with t, particularly in the shared model. A finer characterisation of the Swin corner tokens, including how they evolve after stage 3, is left for future work.

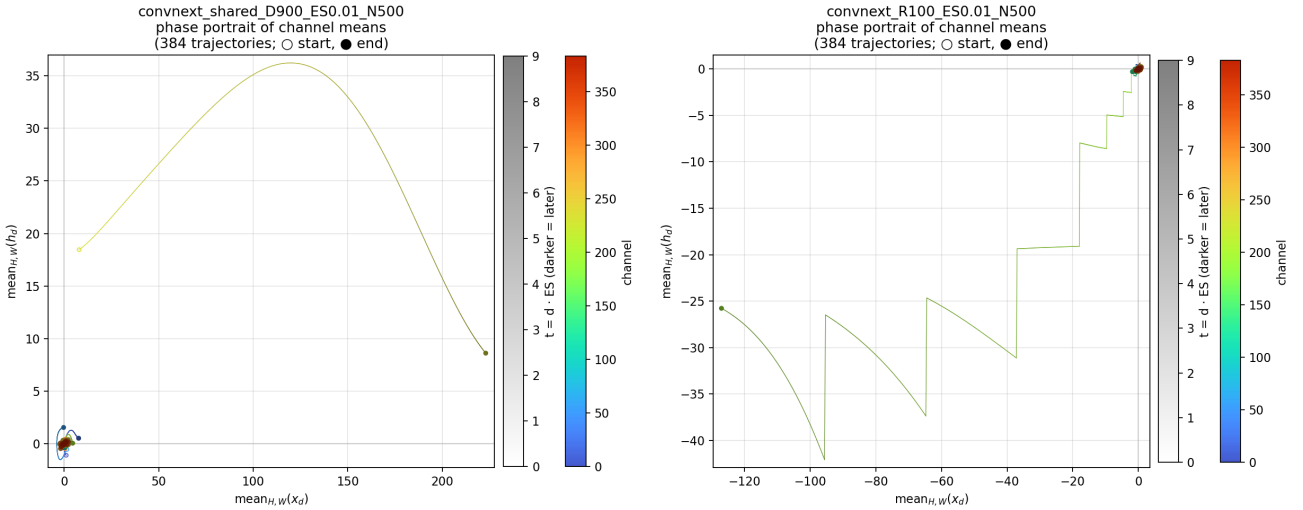


Figure 13. Phase portraits of stage-3 channel means $(\mathrm{mean}_{H,W}(x_d),\ \mathrm{mean}_{H,W}(h_d))$ over $T{=}9$ ($\Delta t{=}0.01$, $N{=}500$ images). **Left:** shared ConvNeXt ($D{=}900$). **Right:** untied ConvNeXt-T ($R{=}100$). One channel reaches spatial means more than an order of magnitude larger than those of the remaining 383 channels, which stay close to the origin.

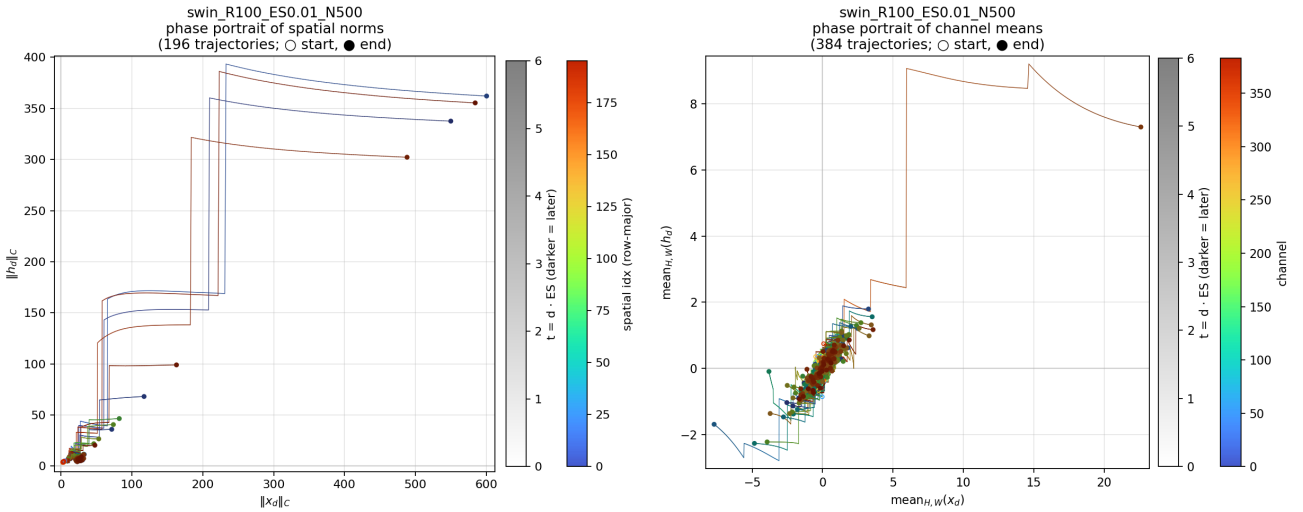


Figure 14. Swin-T stage 3 ($R{=}100$, $\Delta t{=}0.01$, $T{=}6$, $N{=}500$ images). **Left:** one trajectory per spatial position, $(\|x_d\|_C,\ \|h_d\|_C)$; the four outlying trajectories are the corner tokens. **Right:** one trajectory per channel, $(\mathrm{mean}_{H,W}(x_d),\ \mathrm{mean}_{H,W}(h_d))$; one channel grows above the rest in a block-wise sawtooth.

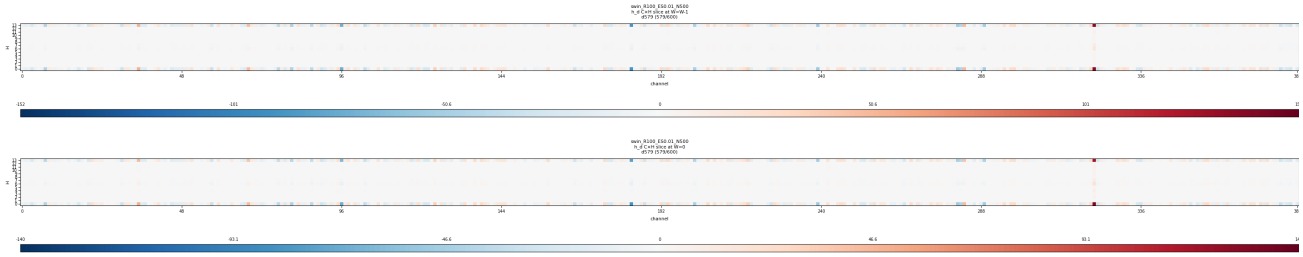


Figure 15. Residual h_d of Swin-T stage 3 at step $d{=}579$ (last block), averaged over 500 images and shown as channel by height slices at the last column (**top**) and at $W{=}0$ (**bottom**). Rows 0 and 13 of each panel are the four corner tokens, which share the same dominant channels and signs.